

11. Libro de ejercicios de transformaciones 2D

Geometría Computacional

Iván Sancho as Unreal Authorized Instructor (UAI)

IMPORTANTE

Todo el código mostrado en este documento de respuestas a los ejercicios de clase es pseudocódigo y sólo representa los pasos necesarios para renderizar cada una de las siguientes transformaciones.

“esat/math.h”

Para el desarrollo de los ejercicios de este libro de transformaciones vamos a valernos, inicialmente, de las funciones que componen la unidad de compilación “math.h” de la librería gráfica de ESAT.

Paralelamente, el alumno puede probar las funciones de su librería matemática para comprobar si realmente las ha programado correctamente y cumplen con el objetivo para las que fueron desarrolladas e ideadas.

```
const double dPI = 3.141592653589793238462643383279502884197169;
const float fPI = 3.141592653589793238462643383279502884197169f;

struct Vec2 { float x, y; };
struct Vec3 { float x, y, z; };
struct Vec4 { float x, y, z, w; };

struct Mat3 { float d[9]; };
struct Mat4 { float d[16]; };

Mat3 Mat3Multiply(const Mat3 &a, const Mat3 &b);
Mat3 Mat3Identity();
Mat3 Mat3Translate(float translate_x, float translate_y);
Mat3 Mat3Rotate(float radians);
Mat3 Mat3Scale(float scale_x, float scale_y);
Mat3 Mat3FromColumns(const float data[9]);
Vec3 Mat3TransformVec3(const Mat3 &a, const Vec3 &v);
Vec2 Mat3TransformVec2(const Mat3 &a, const Vec2 &v);

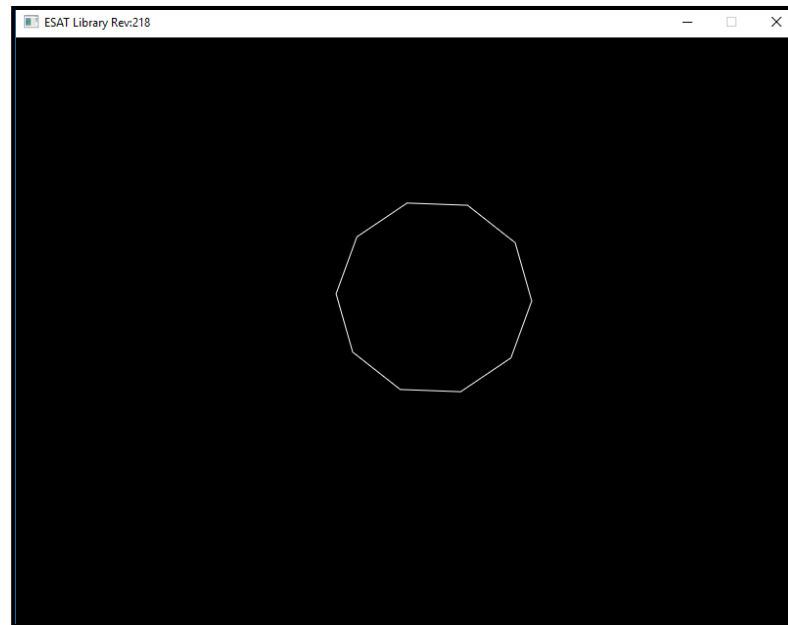
Mat4 Mat4Multiply(const Mat4 &a, const Mat4 &b);
Mat4 Mat4Identity();
Mat4 Mat4Translate(float translate_x, float translate_y, float translate_z);
Mat4 Mat4RotateX(float radians);
Mat4 Mat4RotateY(float radians);
Mat4 Mat4RotateZ(float radians);
Mat4 Mat4Scale(float scale_x, float scale_y, float scale_z);
Mat4 Mat4FromColumns(const float data[16]);
Mat4 Mat4Projection();
Vec4 Mat4TransformVec4(const Mat4 &a, const Vec4 &v);
Vec3 Mat4TransformVec3(const Mat4 &a, const Vec3 &v);

void Mat3TransformRawVec2(const Mat3 &m, const float v_in[2], float v_out[2]);
void Mat3TransformRawVec3(const Mat3 &m, const float v_in[3], float v_out[3]);
void Mat4TransformRawVec3(const Mat4 &m, const float v_in[3], float v_out[3]);
void Mat4TransformRawVec4(const Mat4 &m, const float v_in[4], float v_out[4]);
```

1

1ª Transformación

Representa un círculo de diez vértices que rote en torno a sí mismo en el sentido de las agujas del reloj, que tenga como centro la posición del ratón y que esté escalado 100 unidades.



1ª Transformación: pasos

1. Lo primero a calcular son los puntos de nuestro círculo y los vamos a mantener normalizados entre '- 1' y '1', no vamos a escalarlos ni trasladarlos. Ahora mismo, nuestro círculo es un píxel en la esquina superior izquierda.

```
for (0 ... kNumberVertices) {  
    pos[...].x = cos((kCircunference / kNumberVertices) * i)  
    pos[...].y = sin((kCircunference / kNumberVertices) * i)  
}
```



1ª Transformación: pasos

2. Para poder concatenar matrices en una sola matriz de transformación vamos a comenzar multiplicando cada uno de nuestros puntos por una matriz que va a contener una transformación básica, por lo que la primera operación que afectará a los puntos de nuestro círculo es el escalado.

```
Final_point = S(100.0f, 100.0f) * Initial_point
```



1ª Transformación: pasos

3. Como podemos ver, tenemos nuestro círculo escalado en torno al punto de origen de la pantalla. A continuación, vamos a hacer que rote en sentido de las agujas del reloj valiéndonos de una variable que incremente su valor por “frame”.

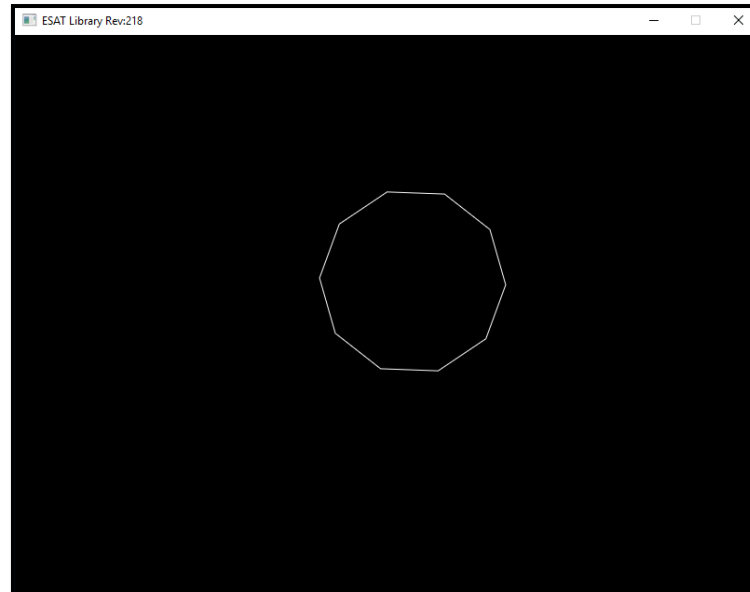
```
Final_point = R(g_angle) * S(100.0f, 100.0f) * Initial_point
```



1ª Transformación: pasos

4. El paso final es aplicar una traslación para cerrar nuestra transformación básica. El valor de esta traslación va a estar en la posición del ratón.

```
Final_point = T(mouse_x, mouse_y) * R(g_angle) * S(100.0f, 100.0f) *  
Initial_point
```



2

2ª Transformación

En base al círculo de diez vértices programado en el ejercicio anterior, utilizar la misma función de pintado para representar las siguientes cuatro figuras con estas características:

Círculo 1: `S(100.0f), R(CW),`
`T(150.0f, 200.0f)`

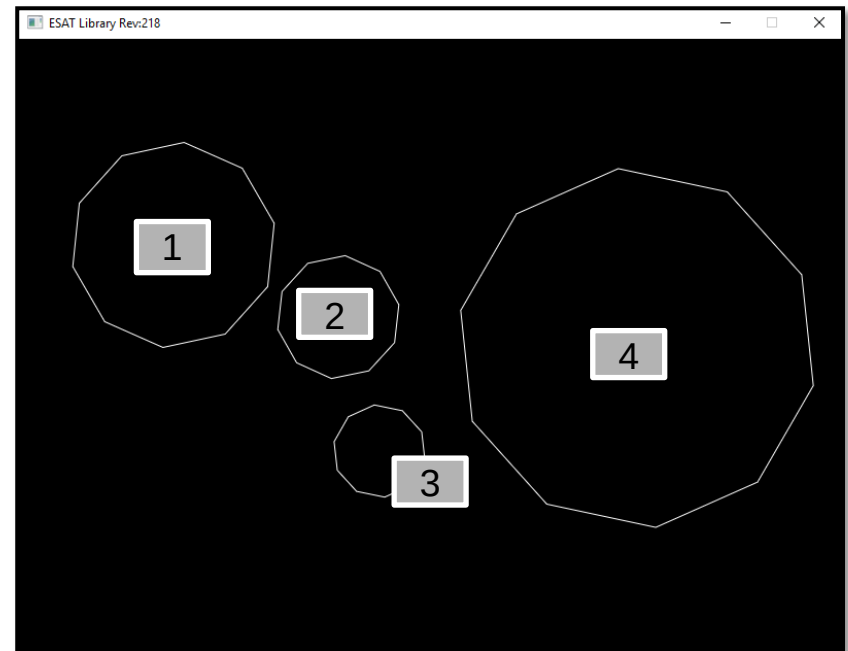
Círculo 2: `S(175.0f), R(CCW),`
`T(600.0f, 300.0f)`

Círculo 3: `S(45.0f), R(CW),`
`T(310.0f, 400.0f)`

Círculo 4: `S(60.0f), R(CCW),`
`T(310.0f, 270.0f)`

*CW = sentido agujas del reloj

*CCW = sentido contrario a las agujas del
reloj



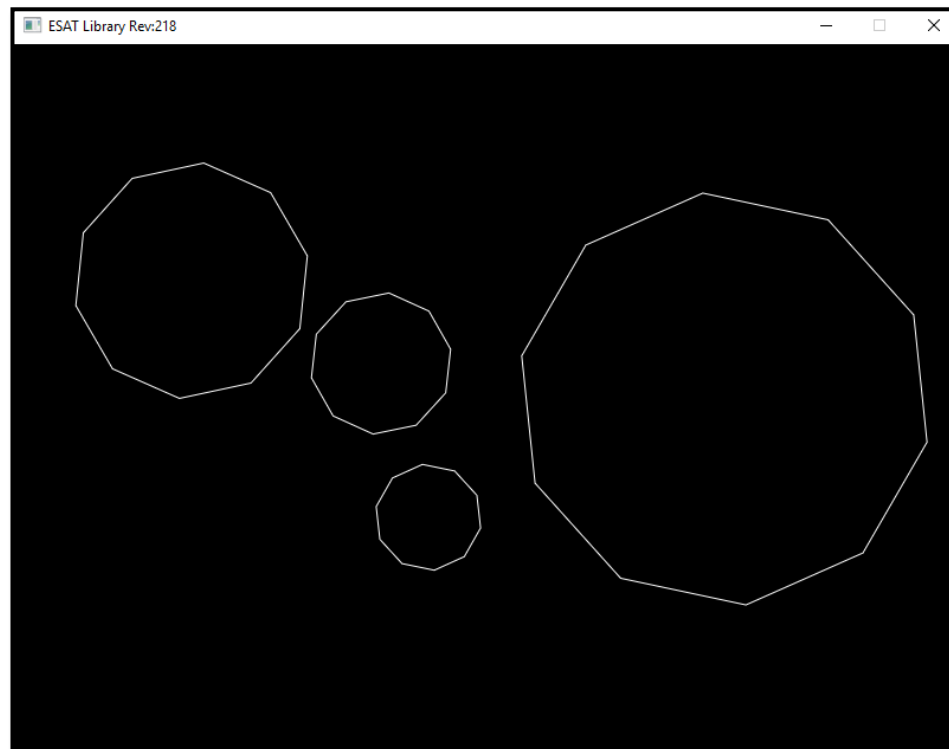
2ª Transformación: pasos

1. Tomando como base el círculo programado en el ejercicio anterior, la clave para poder resolver este ejercicio es programar una función que se encargue de realizar la transformación y pintar los vértices y que reciba, por parámetros, los datos clave: puntos a transformar, radio del escalado, ángulo de rotación y traslación en los dos ejes.

```
void TransformAndDraw(const float* shape_coords, const float radius,  
                     const float tx, const float ty, const float angle){  
    ...  
}
```

2ª Transformación: pasos

2. De este modo, podemos conseguir pintar distintas figuras empleando el mismo conjunto de puntos, simplemente llamando a transformar y pintar varias veces.

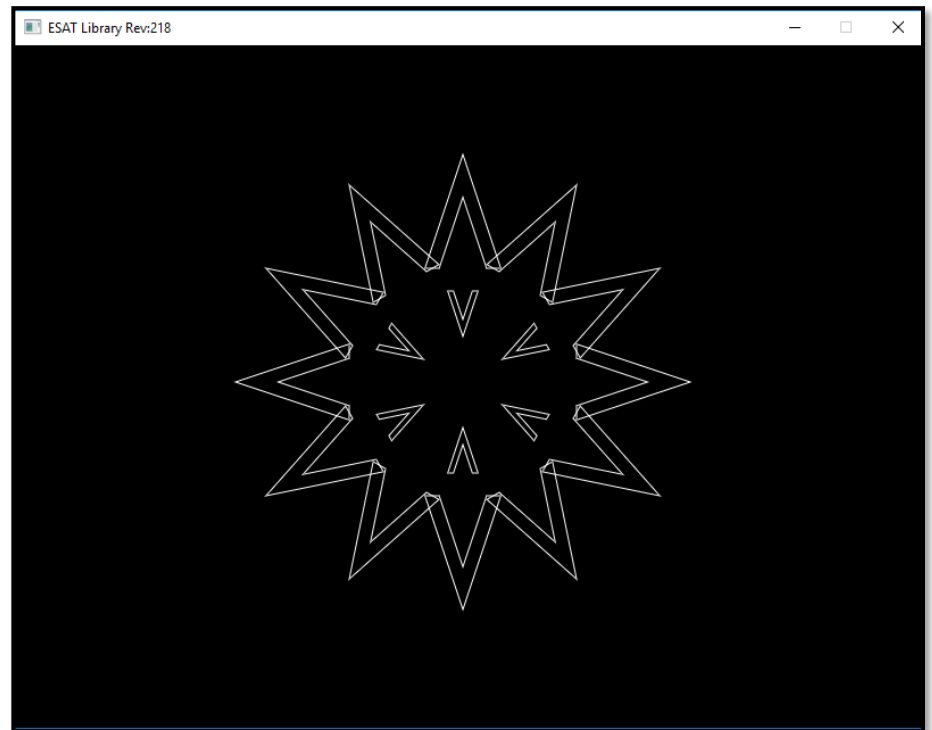


3

3ª Transformación

Dado el siguiente “array” de puntos con coordenadas en dos ejes, homogeneízalos de modo que se encuentren entre -1.0f y 1.0f en los dos ejes y transfórmalos para representar la siguiente figura.

```
float points[14] = {  
    0.0f, 2.0f,  
    - 5.0f, - 8.0f,  
    - 8.0f, - 8.0f,  
    0.0f, 8.0f,  
    8.0f, - 8.0f,  
    5.0f, - 8.0f,  
    0.0f, 2.0f  
};
```

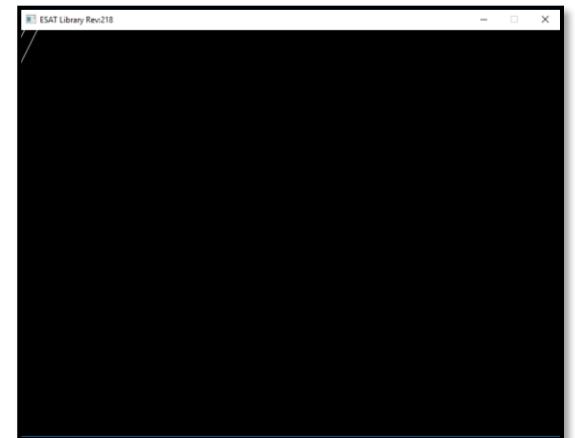


3ª Transformación: pasos

1. En esta composición, lo primero que habría que preguntarse es entre qué máximo homogeneizar los puntos en los dos ejes. Todo depende del rango de valores en el que nos queramos mover. Si nos fijamos, los puntos en coordenadas locales se encuentran en el rango que va de “8.0f” a “8.0f”, así que si tomamos este último valor como máximo, obtendremos puntos entre “-1.0f” y “1.0f”.

De entrada vamos a hacer eso, normalizar entre el máximo en ambos ejes, y vamos a hacer una transformación básica compuesta por un escalado a “50.0f” unidades para ver la figura entorno al punto de origen de la ventana:

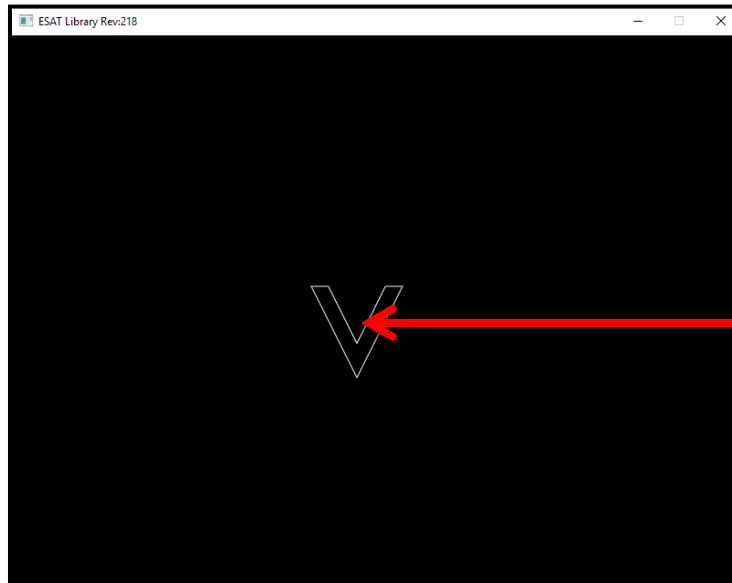
```
Final_point = S(50.0f, 50.0f) * Initial_point
```



3ª Transformación: pasos

2. A continuación, vamos a trasladar nuestra figura a la posición del ratón para poderla ver al completo. Añadimos una traslación a la transformación:

```
Final_point = T(Mouse_X, Mouse_Y) * S(50.0f, 50.0f) * Initial_point
```



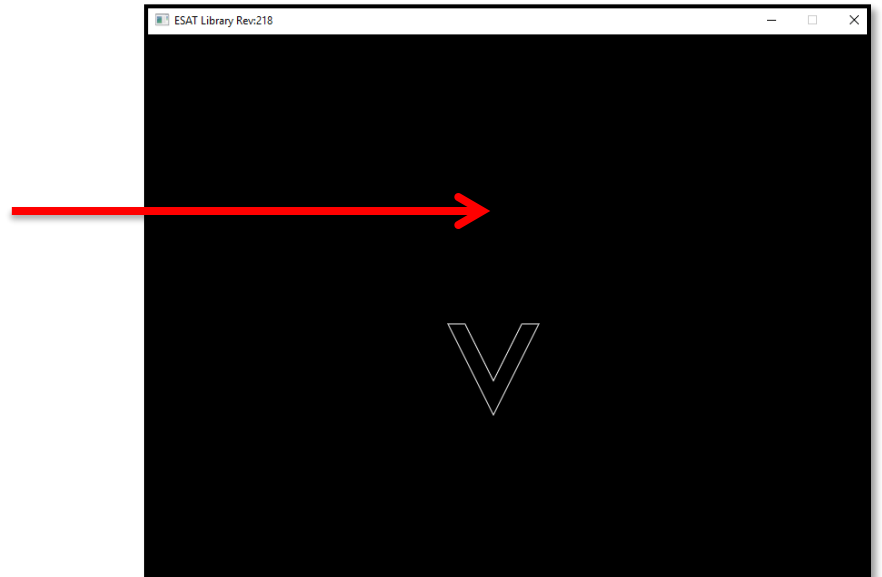
El ratón estaba aquí y este es su centro.

3ª Transformación: pasos

3. Ahora vamos a cambiar su punto de pivotaje. Como tenemos normalizado el tamaño de la figura entre un rango de “- 1.0f, 1.0f”, si queremos desplazar el centro tres radios en el eje de la “Y” tenemos que añadir una traslación inicial:

```
Final_point = T(Mouse_X, Mouse_Y) * S(50.0f, 50.0f) * T(0.0f, 3.0f) *  
Initial_point
```

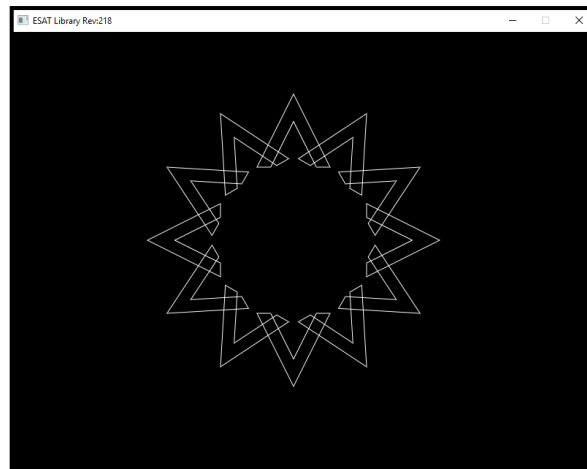
El ratón estaba aquí y
este es el nuevo centro
de la figura.



3ª Transformación: pasos

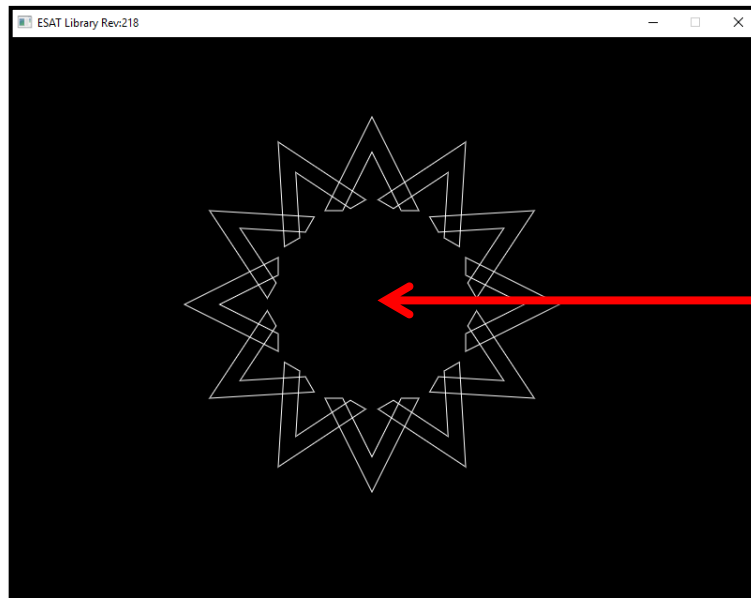
4. Llegados a este punto es hora de dibujar más de una figura y cada una en una posición diferente, dependiendo además del número de figuras a dibujar. En este ejemplo, vamos a realizar doce figuras, así que como hemos cambiado inicialmente el punto de pivotaje con una traslación en el eje de la “Y”, si a continuación rotáramos, los puntos orbitarían en torno a este centro local:

```
Final_point = T(Mouse_X, Mouse_Y) * R((kCircunference / kNumberOuterShapes) * i)  
            * S(50.0f, 50.0f) * T(0.0f, 3.0f) * Initial_point
```



3ª Transformación: pasos

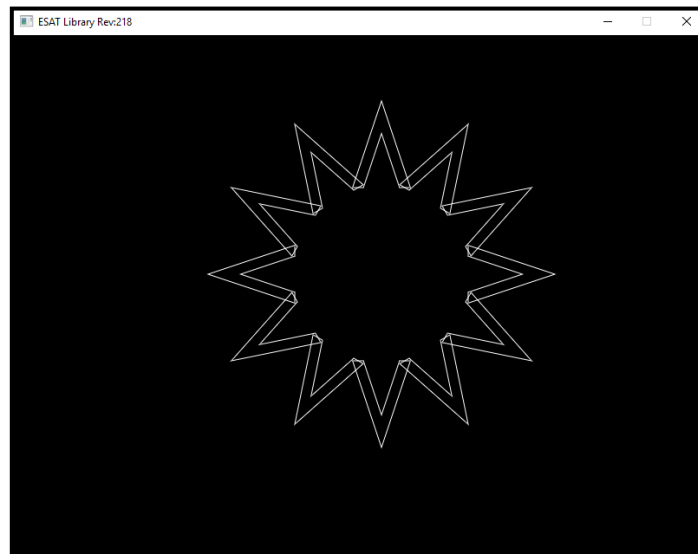
Y bien, ¿por qué hemos añadido la rotación en este punto? Porque el centro de la figura ya ha sido modificado, la hemos escalado también, y si ahora rotamos en un ángulo distinto, lo que vamos a conseguir es que, como ha sucedido, las distintas piezas orbiten en torno al punto de pivotaje. La traslación final va a mover la figura a la posición del ratón.



El ratón se encontraba en este punto y este es el centro de la composición.

3ª Transformación: pasos

5. Como podemos ver, además, tenemos un problema de solapamiento de las figuras. Es algo que solucionamos si, en el punto inicial, el eje de la “X” lo normalizamos entre una cifra mayor a la que estábamos trabajando hasta ahora, en este caso vamos a tomar como cociente orientativo el número de figuras que tenemos en la pantalla. Además, no rompe nada de nuestro planteamiento inicial porque las figuras siguen estando entre “ $-1.0f$, $1.0f$ ” en ambos ejes:

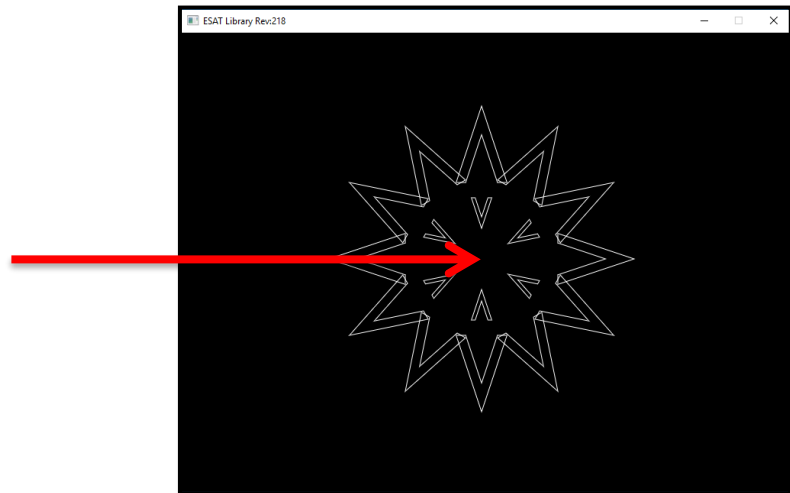


3ª Transformación: pasos

6. Finalmente, para representar las geometrías interiores vamos a seguir el mismo orden de operaciones pero modificando dos datos: la escala es menor, y la primera traslación se encuentra tres unidades hacia arriba en el eje de la “Y”. En este caso vamos a representar seis figuras:

```
Final_point = T(Mouse_X, Mouse_Y) * R((kCircunference / kNumberInnerShapes) * i)  
            * S(20.0f, 20.0f) * T(0.0f, -3.0f) * Initial_point
```

Centro de la figura.

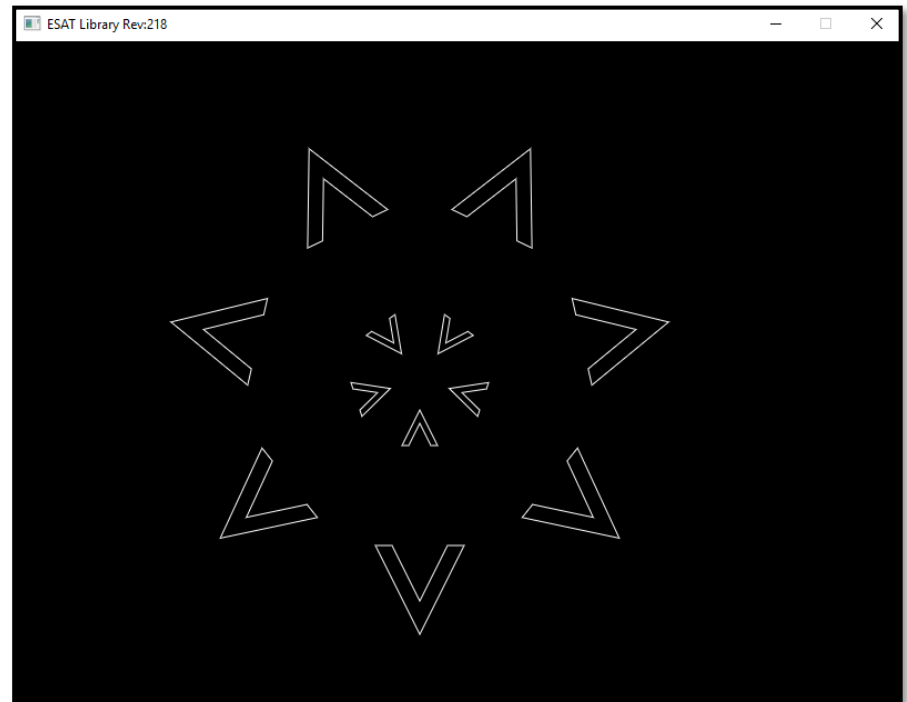


4

4ª Transformación

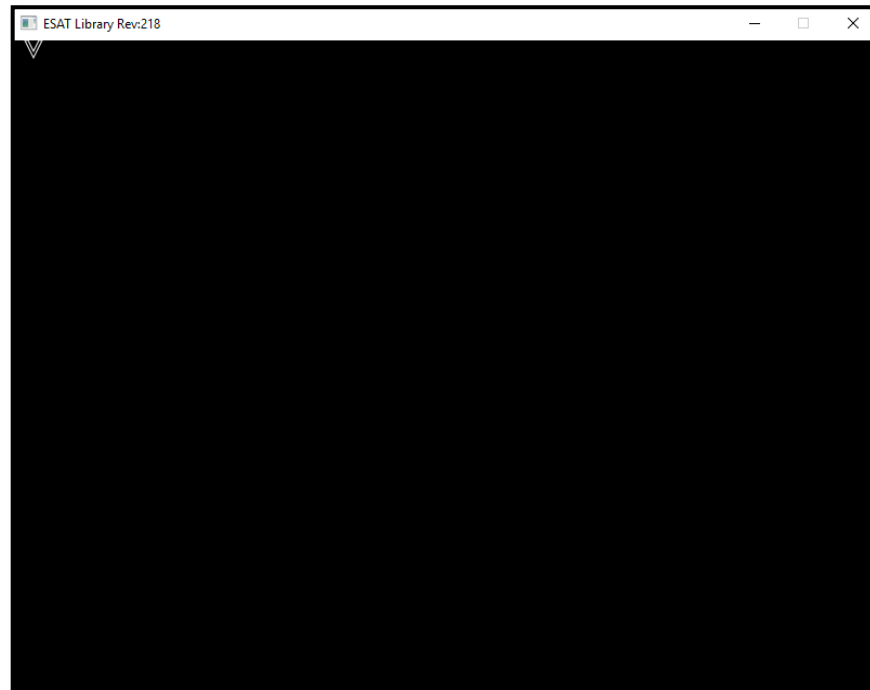
Dado el siguiente “array” de puntos con coordenadas en dos ejes, transfórmalos sin homogeneizar para representar la siguiente figura. Las dimensiones y traslaciones serán aproximadas, no así las rotaciones.

```
float points[14] = {  
    20.0f, 10.0f,  
    15.0f, 0.0f,  
    12.0f, 0.0f,  
    20.0f, 16.0f,  
    28.0f, 0.0f,  
    25.0f, 0.0f,  
    20.0f, 10.0f,  
};
```



4ª Transformación: pasos

1. Si, como en el anterior ejercicio, comenzamos dibujando la figura sin ningún tipo de transformación, se muestra tal y como se ve en la siguiente imagen, con centro en el punto de origen de la ventana:



4ª Transformación: pasos

2. Si queremos comenzar desarrollando el anillo exterior, vamos a comenzar centrando nuestra figura. Para hacer eso hay que observar sus dimensiones para ver que son de “40.0f, 16.0f”, por lo que hay que trasladarla “– 20.0f, – 8.0f” píxeles:

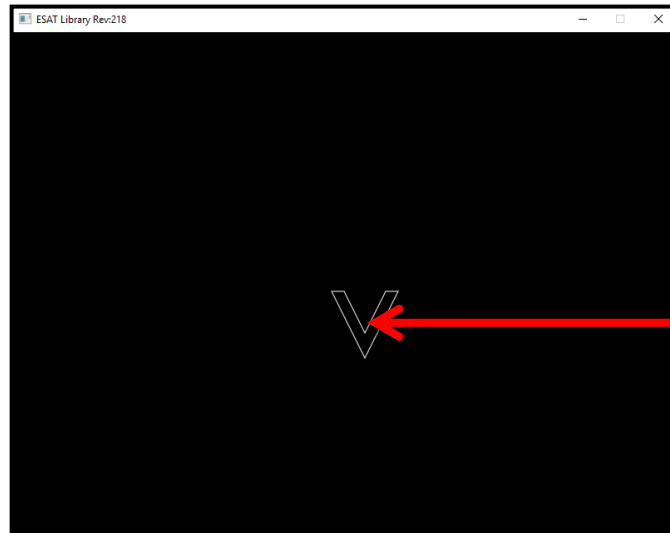
```
Final_point = T(-20.0f, -8.0f) * Initial_point
```



4ª Transformación: pasos

3. Ahora que hemos cambiado el centro de la figura, vamos a escalarla y a trasladarla a la posición del ratón. Hay que tener cuidado escalando una geometría de estas dimensiones y pensar en los valores resultantes de nuestra operación:

```
Final_point = T(Mouse_X, Mouse_Y) * S(5.0f, 5.0f) * T(-20.0f, -8.0f) *  
Initial_point
```



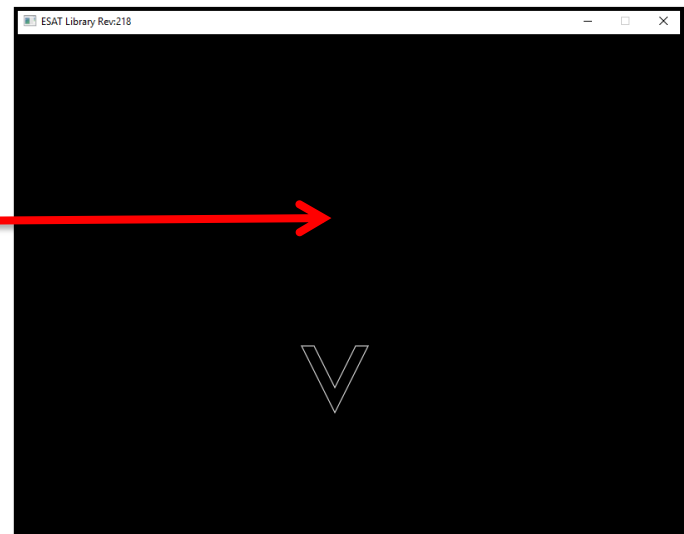
**El ratón estaba aquí y
este es su centro.**

4ª Transformación: pasos

4. Como podemos ver, la traslación inicial nos servía para cambiar el punto de pivotaje pero no para cambiar nuestros propósitos. Si queremos colocar varias figuras orbitando alrededor de un centro, deberíamos cambiar el valor del eje de la “Y” en la primera traslación.

```
Final_point = T(Mouse_X, Mouse_Y) * S(5.0f, 5.0f) * T(-20.0f, 30.0f) *  
Initial_point
```

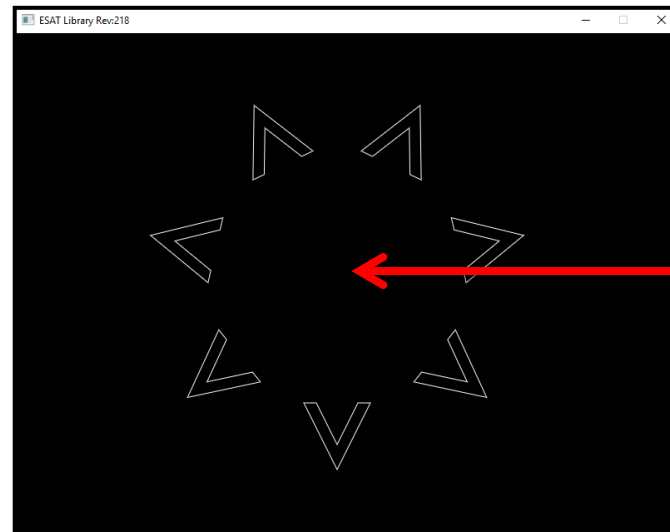
El ratón estaba aquí y
este es su centro.



4ª Transformación: pasos

5. De este modo, si queremos representar más figuras alrededor del nuevo centro, podemos añadir una rotación previa a la última traslación para que orbiten.

```
Final_point = T(Mouse_X, Mouse_Y) * R((kCircunference / kNumberOuterShapes) * i)  
            * S(5.0f, 5.0f) * T(-20.0f, 30.0f) * Initial_point
```



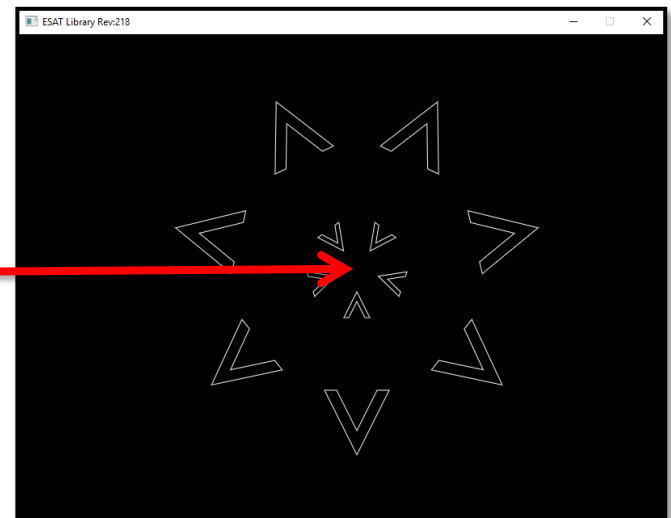
**El ratón estaba aquí y
este es su centro.**

4ª Transformación: pasos

6. En este punto final, podemos comenzar a calcular la transformación para las figuras interiores. Los pasos son idénticos excepto un par de excepciones: tenemos que añadir una rotación inicial de 180° (3.1415 in radians) y la traslación en el eje de la "X" será de "20.0f".

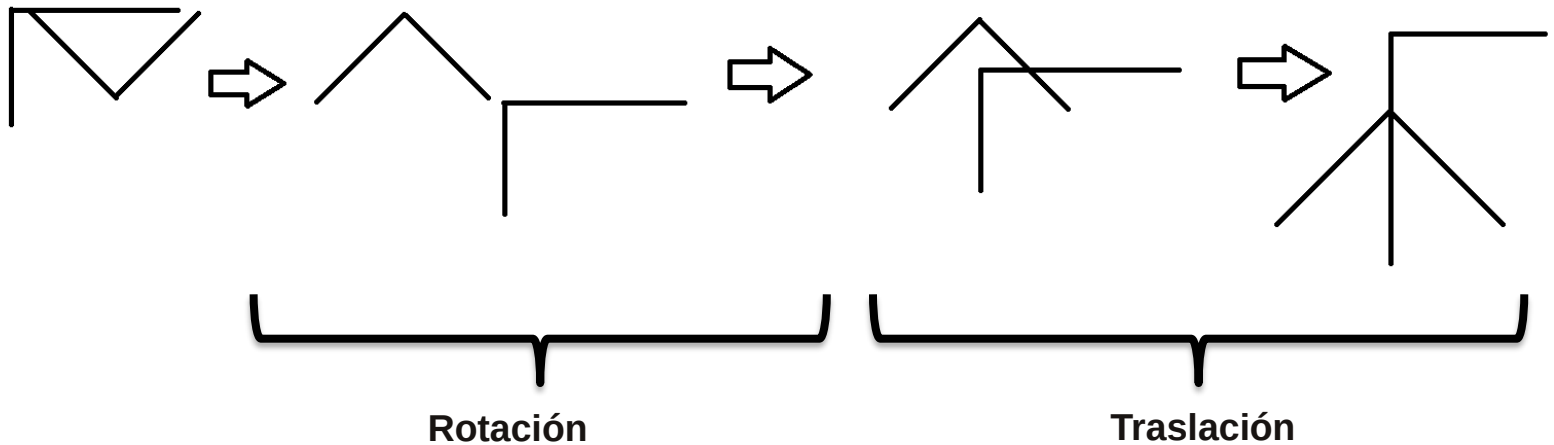
```
Final_point = T(Mouse_X, Mouse_Y) * R((kCircunference / kNumberInnerShapes) * i)
              * S(2.0f, 2.0f) * T(20.0f, 30.0f) * R(3.1415) * Initial_point
```

El ratón estaba aquí y
este es su centro.



4ª Transformación: pasos

¿Por qué hemos hecho estos cambios? Sólo pensad en la siguiente secuencia e intentad representarla en un papel:

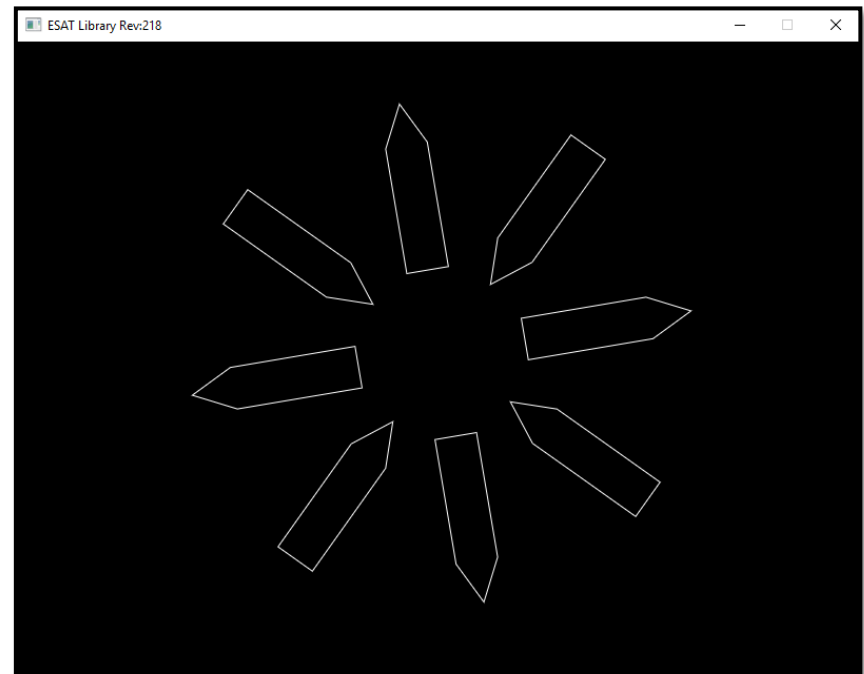


5

5ª Transformación

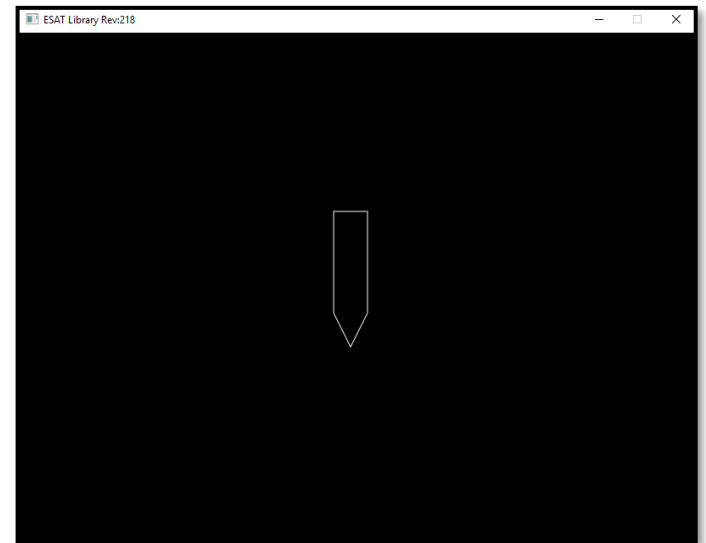
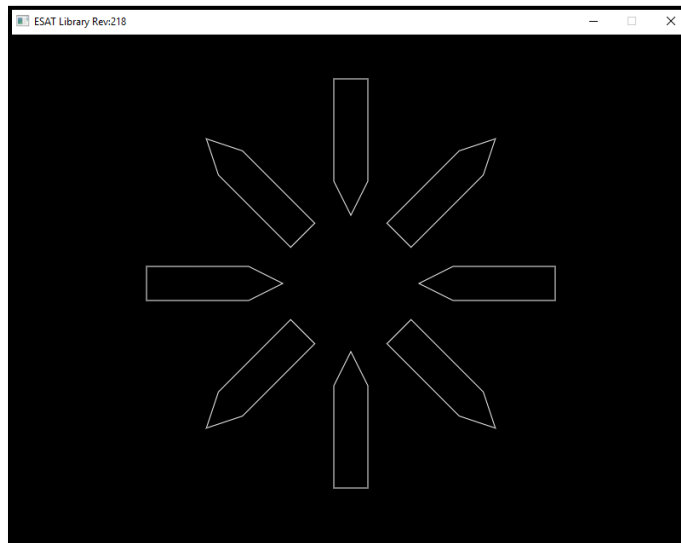
Dados los siguientes puntos, transfórmalos para representar la siguiente composición. Las figuras rotan en sentido de las agujas del reloj, están escaladas un total de “160” unidades y tienen como centro la posición del ratón.

```
float points[10] = {  
    0.0f, 0.0f,  
    0.0f, 0.75f,  
    0.125f, 1.0f,  
    0.25f, 0.75f,  
    0.25f, 0.0f,  
};
```



5ª Transformación: pasos

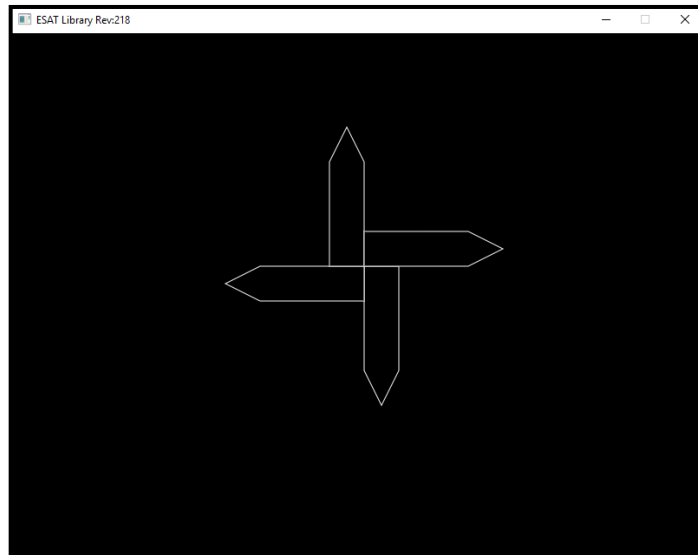
1. Bien, en primer lugar, piensa cómo sería el conjunto de figuras trasladadas y escaladas pero sin tener el efecto de rotación... y piensa en la forma de una única figura individual.



5ª Transformación: pasos

2. Vamos a plantear la composición en un par de grupos: las figuras verticales y horizontales por una parte y las diagonales por la otra. Vamos a comenzar con las cuatro figuras verticales / horizontales con una transformación muy básica:

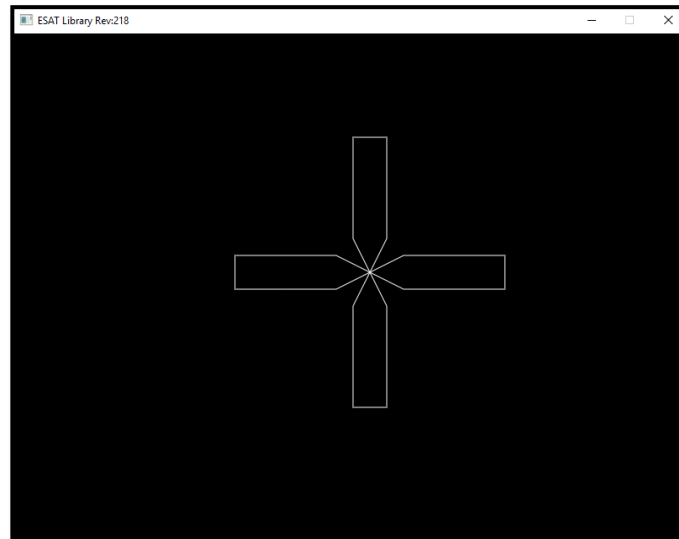
```
Final_point = T(Mouse_X, Mouse_Y) * R((kCircunference / kNumberInnerShapes) * i)  
              * S(160.0f, 160.0f) * Initial_point
```



5ª Transformación: pasos

3. Vamos a aplicar una pequeña traslación inicial para lograr que todas las figuras colapsen en el mismo punto.

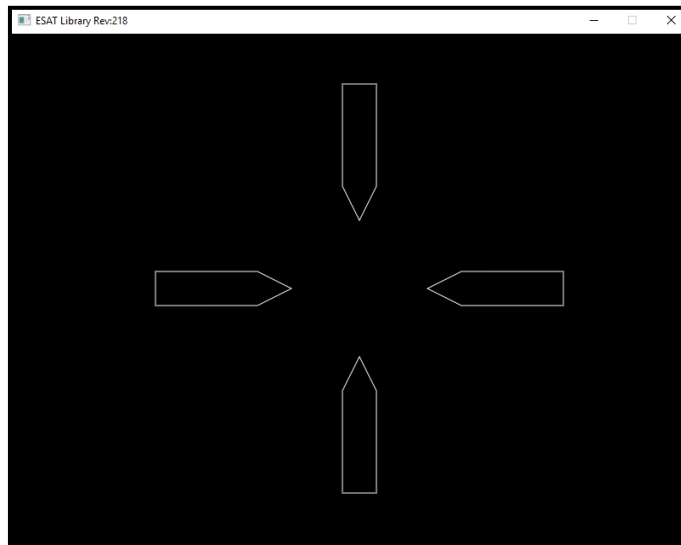
```
Final_point = T(Mouse_X, Mouse_Y) * R((kCircunference / kNumberInnerShapes) * i)  
            * S(160.0f, 160.0f) * T(- 0.125f, - 1.0f) * Initial_point
```



5ª Transformación: pasos

4. Como se puede ver, hemos centrado las figuras en un mismo punto común y, si quisiéramos que tomaran más distancia, aplicaríamos un offset en el eje de la “Y”:

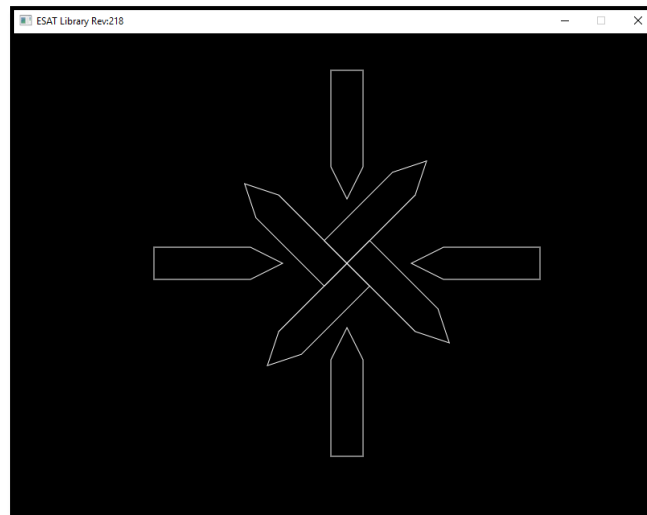
```
Final_point = T(Mouse_X, Mouse_Y) * R((kCircunference / kNumberInnerShapes) * i)  
            * S(160.0f, 160.0f) * T(- 0.125f, - 1.0f - 0.5f) * Initial_point
```



5ª Transformación: pasos

5. Ahora, vamos a hacer lo mismo con el otro grupo de figuras: comenzamos programando una transformación básica en la que, en este caso, vamos a añadir un “offset” a la rotación para colocar cada figura en el lugar correcto:

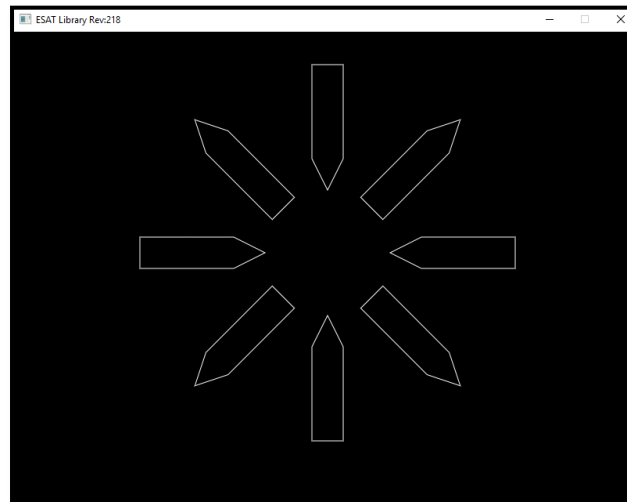
```
Final_point = T(Mouse_X, Mouse_Y) * R((kCircunference / kNumberInnerShapes) * i  
+ ((kCircunference / kNumberFigures) * 0.5f)) * S(160.0f, 160.0f) * Initial_point
```



5ª Transformación: pasos

6. Como se puede observar, tenemos el mismo problema que con el otro grupo de figuras, así que vamos a cambiar el punto de pivotaje.

```
Final_point = T(Mouse_X, Mouse_Y) * R((kCircunference / kNumberInnerShapes) * i  
      + ((kCircunference / kNumberFigures) * 0.5f)) * S(160.0f, 160.0f) *  
      * T(0.0f, - 0.5f) * Initial_point
```

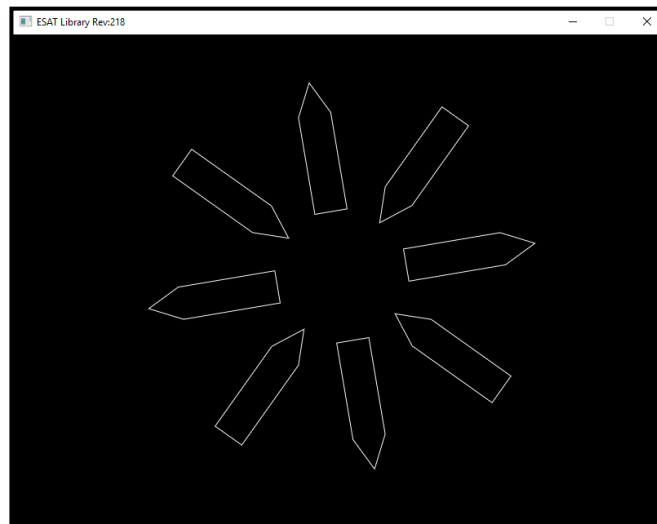


5ª Transformación: pasos

7. Si queremos rotar cada figura, sólo hay que añadir un pequeño “offset” a la rotación de cada tipo de “transform”.

→ $R((kCircunference / kNumberInnerShapes) * i + ((kCircunference / kNumberFigures) * 0.5f) + esat::Time() * 0.001f)$

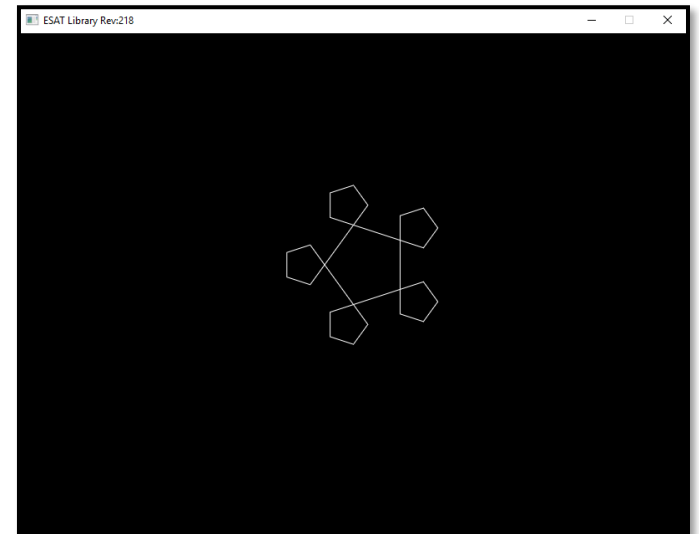
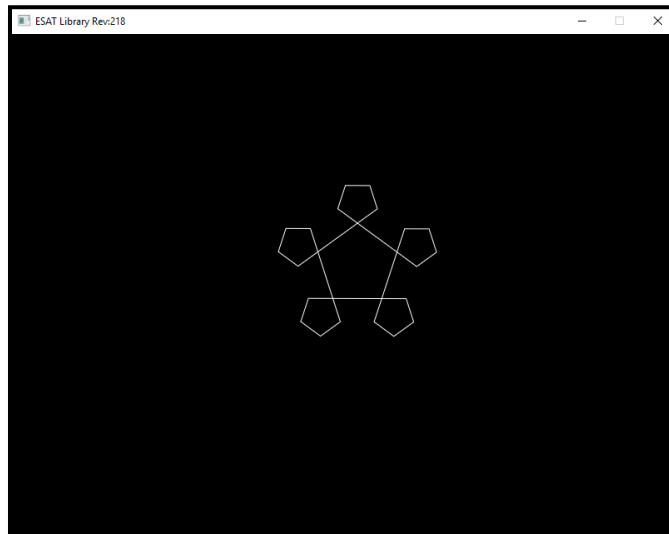
→ $R((kCircunference / kNumberInnerShapes) * i + esat::Time() * 0.001f)$



6

6ª Transformación

Calcula un pentágono (circunferencia de cinco vértices) y, en cada punto, dibuja la misma figura pero con una escala menor. La figura rota en sentido de las agujas del reloj y ejecuta un efecto de “flip – flop” con el fondo de la pantalla para darse el reverso. La figura grande está escalada “50” unidades, y en las pequeñas el escalado es de “20” unidades.



6ª Transformación: pasos

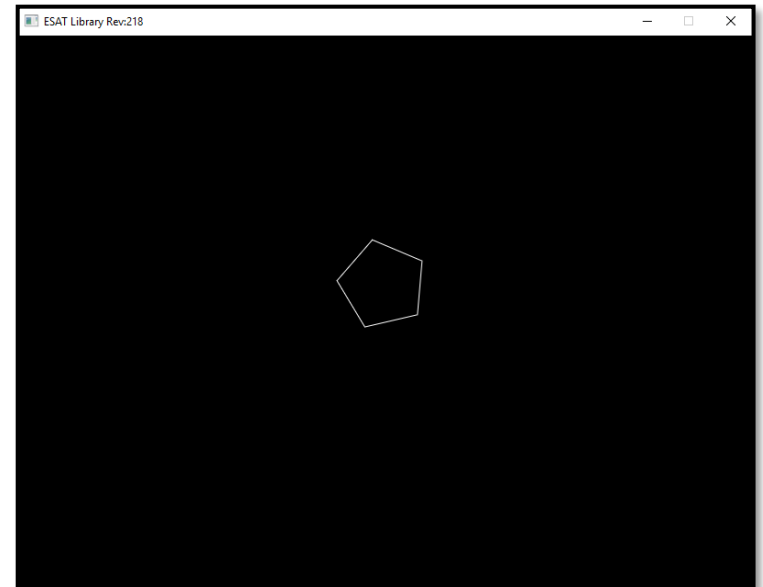
1. Primero de todo, tenemos que calcular los puntos de nuestra figura en un método “Init” que se llame una única vez antes del bucle principal de simulación. En nuestro ejemplo, tenemos que calcular círculos de sólo cinco vértices y, como cada uno está compuesto por un par de coordenadas en ambos ejes, vamos a valernos de las funciones trigonométricas “sin(...)” y “cos(...)” para calcularlos.

```
for (int i = 0; i < kNumberVertices; ++i) {  
    (*shape_coords)[i * 2] = cos((kCircunference / kNumberVertices) * i)  
    (*shape_coords)[i * 2 + 1] = sin((kCircunference / kNumberVertices) * i)  
}
```

6ª Transformación: pasos

2. Una vez que hemos calculado la figura original, como las funciones trigonométricas ya nos devuelven puntos normalizados, vamos a proceder a dibujar la misma figura con una transformación diferente cada vez que queramos mostrar una nueva figura. Con la figura grande, la principal, tenemos que pensar en una transformación básica compuesta por escala, rotación y traslación.

```
Final_point = T(Mouse_X, Mouse_Y) *  
              R(Time() * 0.0001f) *  
              S(50.0f, 50.0f) *  
              Initial_point
```



6ª Transformación: pasos

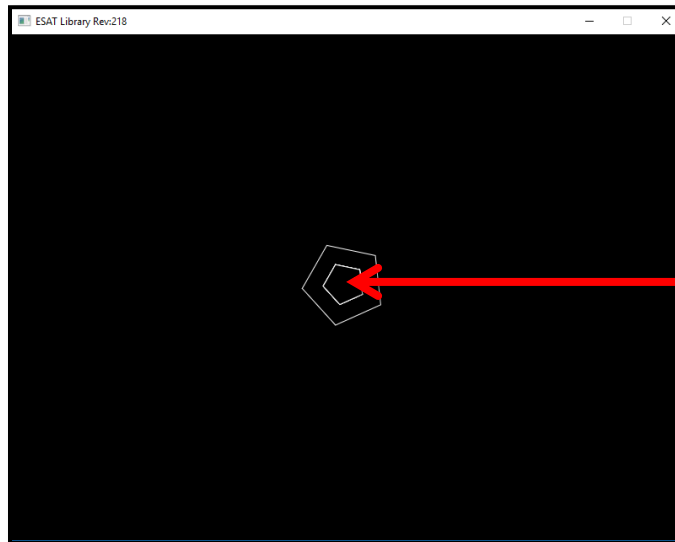
3. En este punto, si queremos dibujar el resto de las figuras, tenemos que pintar tantas como vértices tenga la figura principal. La transformación de cada figura debería “heredar” de la transformación de la principal, que actuará como “base”. Ahora mismo, todas las figuras serán dibujadas en la misma posición, con la misma escala y la misma rotación.

```
for (0 ... a ... kNumberOfVertices) {  
    Final_point = base_transform *  
    Initial_point  
}
```

6ª Transformación: pasos

4. Vamos a comenzar añadiendo la primera operación a la transformación de cada figura hija: un escalado. Como todas ellas heredan de la transformación de la figura principal, si queremos reducir su tamaño a la mitad de la de la figura base, vamos a aplicar un escalado de “0.5f” en los dos ejes.

```
for (0 ... a ... kNumberOfVertices) {  
    Final_point = base_transform * S(0.5f, 0.5f) * Initial_point }  
}
```



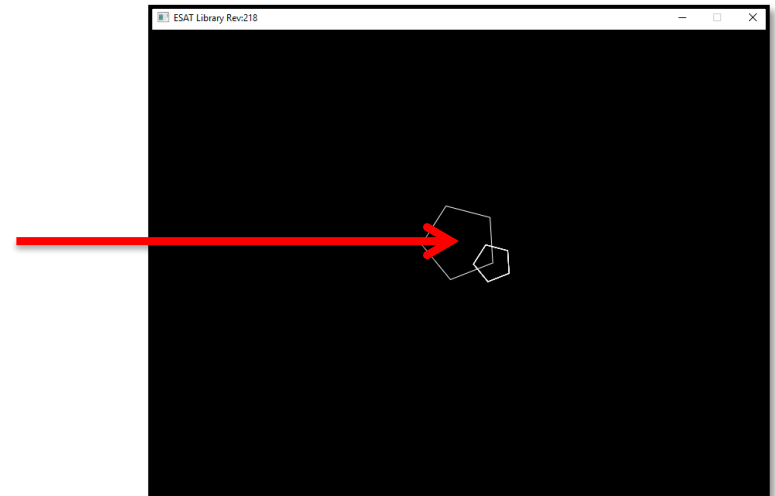
El ratón estaba aquí, este es el centro de la composición.

6ª Transformación: pasos

5. Una vez que hemos cambiado el tamaño de cada figura hija, el siguiente paso podría ser trasladarlas. Como nuestras figuras heredan de la transformación de la principal, tenemos que trabajar con la transformación previa en términos unitarios. Si queremos trasladar nuestra figura la distancia del radio, podemos realizar una traslación básica de $(1.0f, 0.0f)$.

```
for (0 ... a ... kNumberOfVertices) {  
    Final_point = base_transform * T(1.0f, 0.0f) * S(0.5f, 0.5f) * Initial_point  
}
```

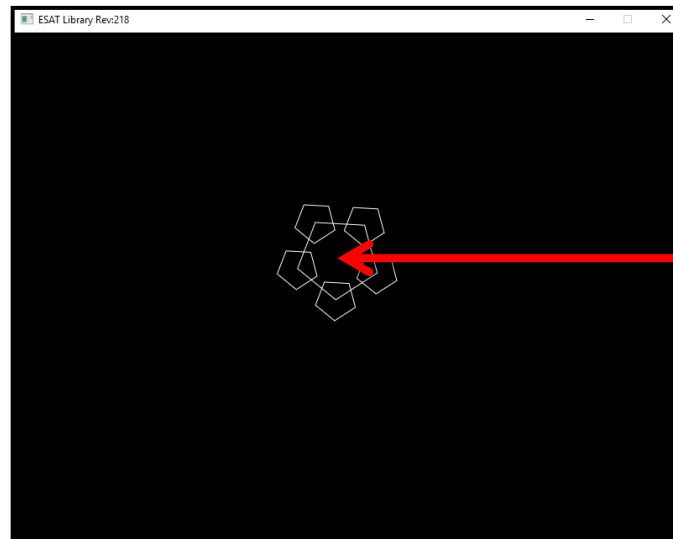
El ratón estaba aquí, este es el centro de la composición.



6ª Transformación: pasos

6. Tenemos todavía un problema: todas las figuras están en la misma posición. Ahora es cuando añadiremos una rotación final para lograr que orbiten en torno al centro:

```
for (0 ... a ... kNumberOfVertices) {  
  
    Final_point = base_transform * R((Circunference / kNumberVertices) * i) *  
    T(1.0f, 0.0f) * S(0.5f, 0.5f) * Initial_point  
  
}
```



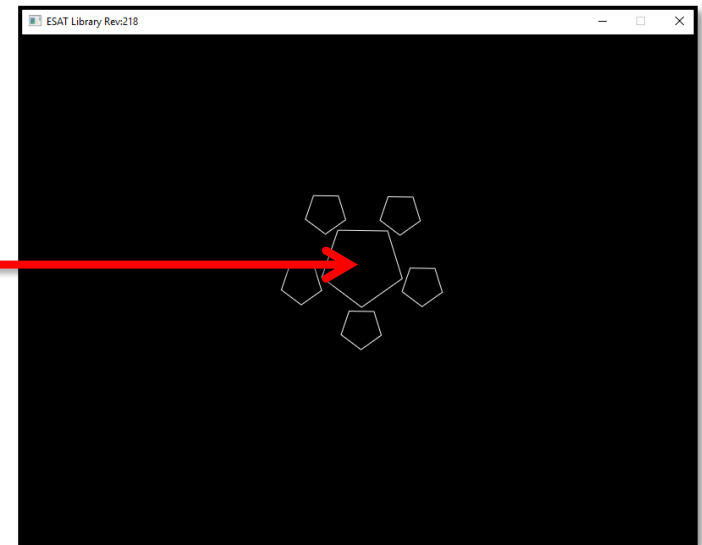
El ratón estaba aquí, este es el centro de la composición.

6ª Transformación: pasos

7. Podemos ajustar la traslación respecto a la figura base estableciéndola en unos valores de “1.5f, 0.0f”, el equivalente a radio y medio de la figura base:

```
for (0 ... a ... kNumberOfVertices) {  
    Final_point = base_transform * R((Circunference / kNumberVertices) * i) *  
        T(1.5f, 0.0f) * S(0.5f, 0.5f) * Initial_point  
}
```

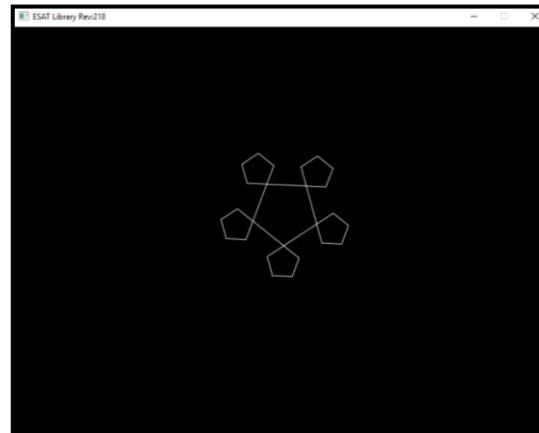
El ratón estaba aquí, este es el centro de la composición.



6ª Transformación: pasos

8. Adicionalmente, si queremos ajustar los vértices de cada figura hija respecto a cada vértice de la figura base, podemos añadir una rotación inicial previa a toda la transformación:

```
for (0 ... a ... kNumberOfVertices) {  
    Final_point = base_transform * R((Circunference / kNumberVertices) * i) *  
        T(1.5f, 0.0f) * S(0.5f, 0.5f) * R(Circunference /  
        kNumberVertices) * Initial_point  
}
```



6ª Transformación: pasos

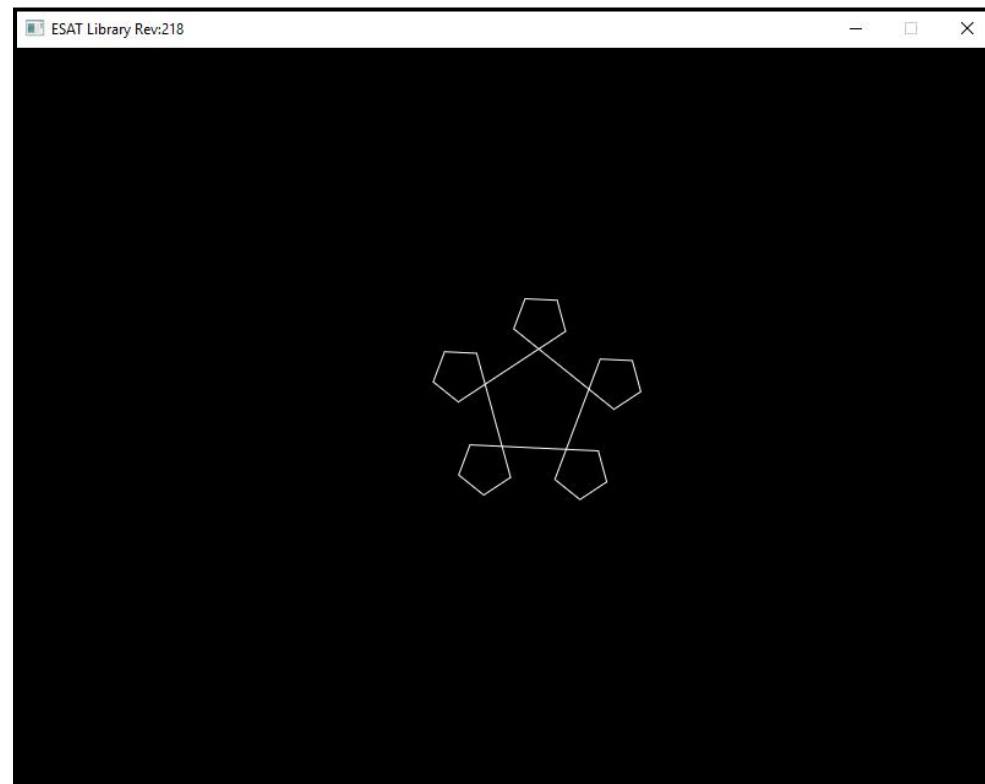
9. Finalmente, para añadir el efecto de reverso con el fondo, vamos a realizar una última modificación a la transformación de la figura principal en el que vamos a modificar el escalado: si el valor original era igual a “50.0f”, ahora nuestro objetivo es obtener un rango de valores entre “- 50.0f” y “50.0f”.

Las funciones trigonométricas seno y coseno podrían aportarnos estos valores. En este caso en concreto, vamos a emplear la función coseno para que la figura empiece grande y vaya dando el reverso. Conseguiremos este efecto si multiplicamos el coseno de una variable auto incrementativa que tomemos como si fueran los radianes por la escala que queremos:

```
base_matrix = T(Mouse_X, Mouse_Y) * R(Time() * 0.0001f) *  
S(cos(Time() * 0.001f) * 50.0f, cos(Time() * 0.001f) * 50.0f)
```

6ª Transformación: pasos

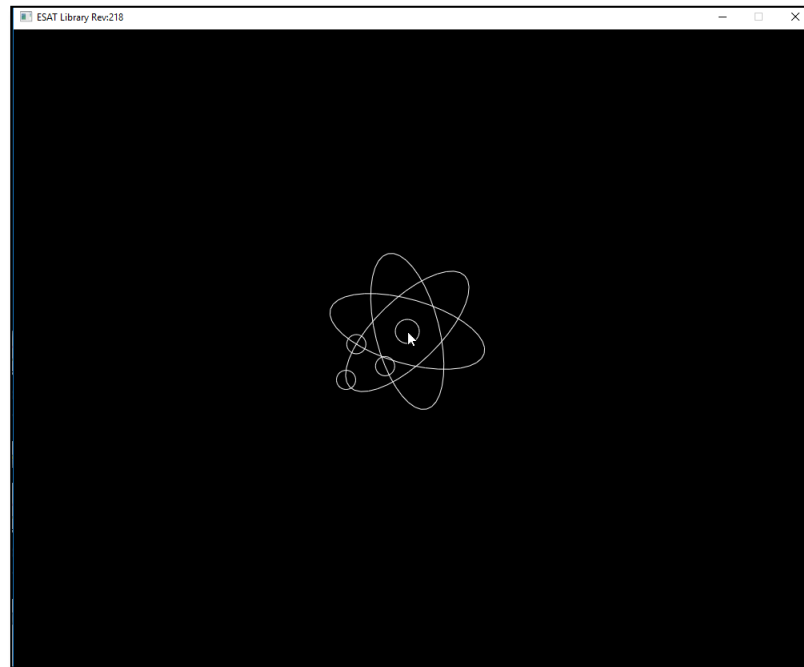
De este modo, conseguimos la composición final:



7

7ª Transformación

Calcula un círculo básico de 36 vértices y realiza la siguiente composición, dibujando siempre el mismo círculo, pero con una transformación diferente cada vez. Las figuras rotarán en el sentido de las agujas del reloj.



7ª Transformación: pasos

1. En primer lugar, vamos a comenzar calculando un círculo básico como hemos hecho hasta ahora:

```
for (0 ... kNumberVertices) {  
    pos[...].x = cos((kCircunference / kNumberVertices) * i)  
    pos[...].y = sin((kCircunference / kNumberVertices) * i)  
}
```



7ª Transformación: pasos

2. Una vez que tenemos precalculado el círculo, vamos a emplearlo para pintar todas las figuras. Podemos comenzar por el círculo pequeño ya que sólo está compuesto por una transformación básica de escalado y traslación.

```
Final_point = T(Mouse_X, Mouse_Y) * S(10.0f, 10.0f) * Initial_point
```



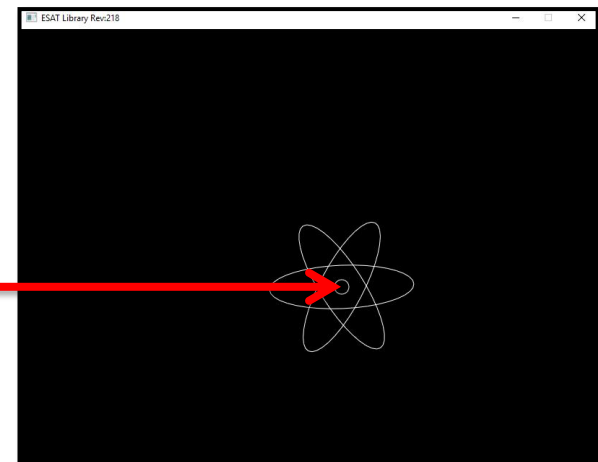
El ratón estaba aquí, este es el centro de la composición.

7ª Transformación: pasos

3. El proceso para pintar las órbitas es similar pero con una transformación básica compuesta por escalado, rotación y traslación. Para dar a cada figura una rotación diferente, tenemos que dividir la circunferencia entre el número de figuras a crear (tres en este caso) y, si queremos añadir una rotación en torno a su centro, tenemos que trabajar con una variable que se incremente en cada “frame”.

```
Final_point = T(Mouse_X, Mouse_Y) * R((kCircunference / kNumberFigures) * i +  
esat::Time() * 0.001f) * S(100.0f, 30.0f) * Initial_point
```

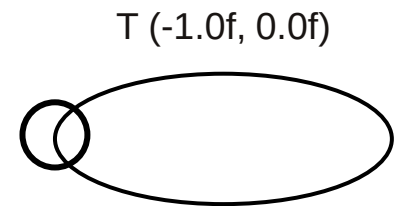
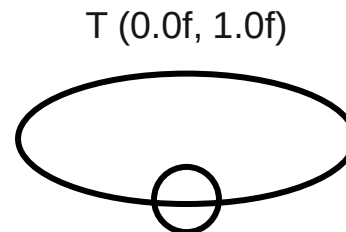
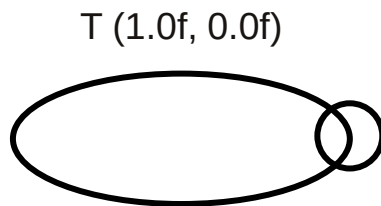
El ratón estaba aquí, este es el centro de la composición.



7ª Transformación: pasos

4. Ahora, si queremos añadir el resto de las esferas, tenemos que detenernos a pensar cómo emplear la transformación de las órbitas para heredar de ellas. Vamos a añadir una pequeña matriz compuesta por traslación y escalado: simularemos la circunferencia en la traslación y, para ello, emplearemos las funciones trigonométricas seno y coseno, y con el escalado “corregiremos” el escalado de la órbita base para crear figuras esféricas.

Para trasladar cada esfera en su órbita, tenemos que pensar en términos unitarios. Cada órbita va a actuar como base de su respectiva esfera: no importa el escalado, la rotación o la traslación de cada una de ellas porque hemos de pensar que cada círculo heredará de ellas y, por lo tanto, tomará las dimensiones de su órbita base como unitarias. Por ejemplo:



7ª Transformación: pasos

5. Si programamos lo que acabamos de plantear, emplearemos las funciones “cos” y “sin” en la traslación con un valor auto incrementativo como el tiempo para jugar con la posición de los planetas en las órbitas.

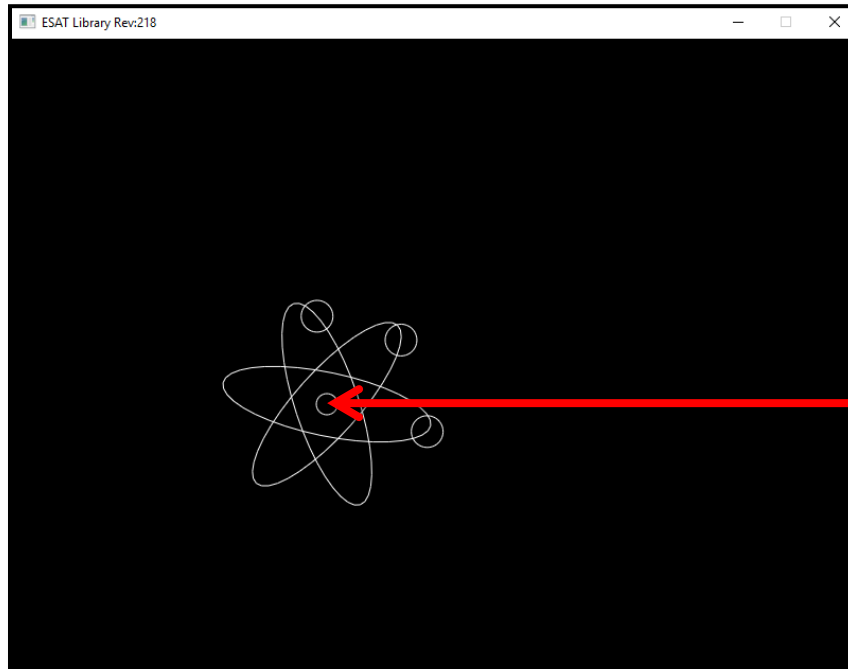
Para el escalado, como hay que anular el de la matriz base, dividimos “1” entre el escalado del base en los dos ejes para anular la herencia del escalado, y a continuación lo multiplicamos por el nuevo escalado que queramos ponerle (“12” por ejemplo).

En último lugar, multiplicamos por la transformación de la órbita para heredar sus valores:

```
Final_point = base_transform * T(cos(Time() * 0.001f), sin(Time() * 0.001f)) *  
                S(1.0f / 100.0f * 12.0f, 1.0f / 30.0f * 12.0f) * Initial_point
```

7ª Transformación: pasos

```
Final_point = base_transform * T(cos(Time() * 0.001f), sin(Time() * 0.001f)) *  
                S(1.0f / 100.0f * 12.0f, 1.0f / 30.0f * 12.0f) * Initial_point
```

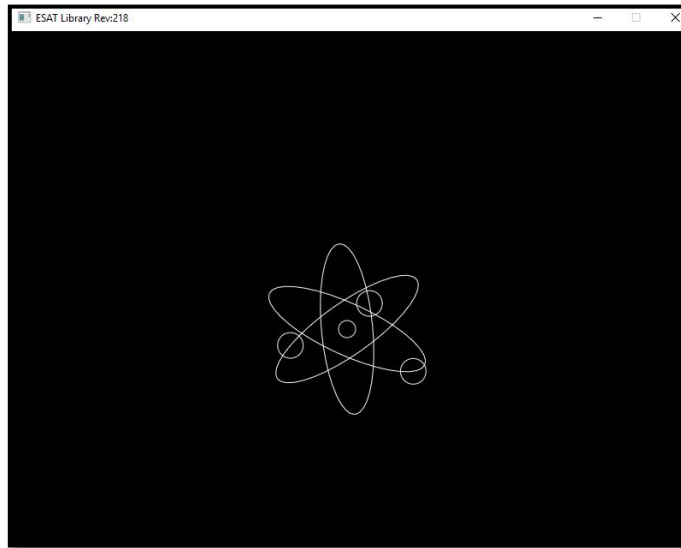


El ratón estaba aquí, este es el centro de la composición.

7ª Transformación: pasos

6. Finalmente, podemos añadir distintas velocidades a cada neutrón multiplicando el input de la traslación de “cos” y “sin” por un número que sea:

```
Final_point = base_transform * T(cos(Time() * 0.001f * speed), sin(Time() *  
                                0.001f * speed)) * S(1.0f / 100.0f * 12.0f, 1.0f / 30.0f *  
                                12.0f) * Initial_point
```



8

8ª Transformación

Programa el siguiente sistema estelar. Pista: cada estrella está rotando en torno a sí misma y está rotando alrededor de sí misma y también orbitando a otra estrella, todo en el sentido de las agujas del reloj y con una velocidad diferente.



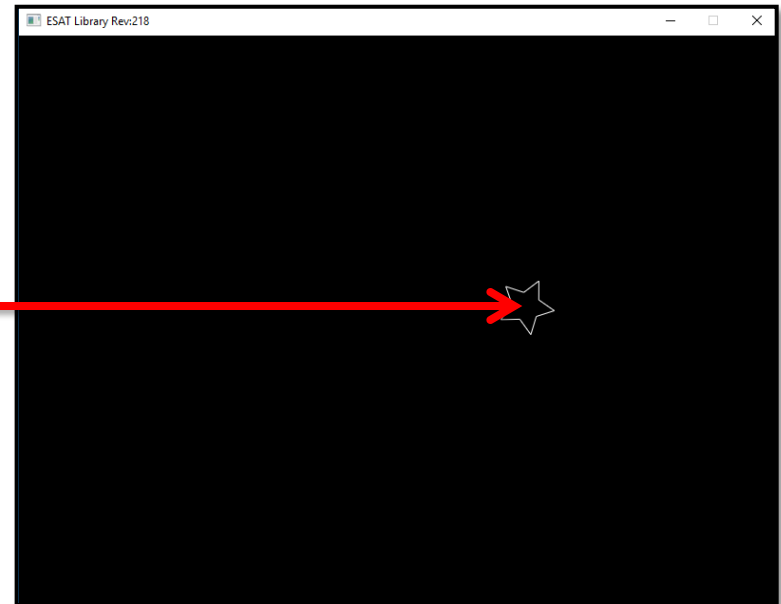
[illegible]

8ª Transformación: pasos

2. Pensemos en una transformación básica para la estrella “base”:

```
Final_point = T(Mouse_X, Mouse_Y) * R(esat::Time() * 0.001f) * S(30.0f, 30.0f) *  
Initial_point
```

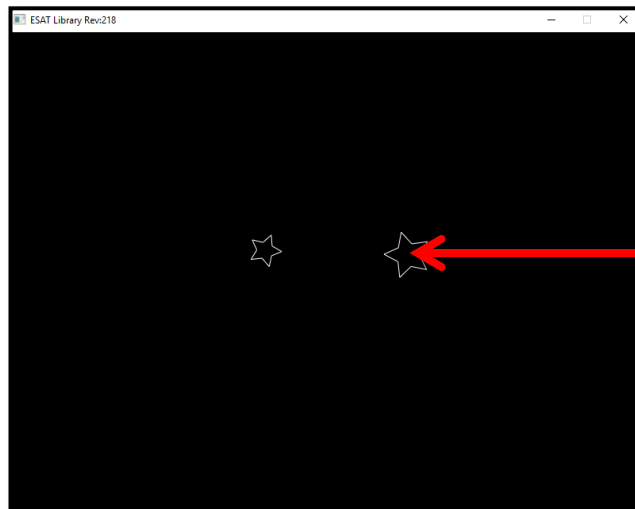
El ratón estaba aquí, este es
el centro de la composición.



8ª Transformación: pasos

3. Para mostrar la segunda estrella, la estrella “hija”, tenemos que heredar de la transformación de la estrella “base”. Todas las operaciones de esta nueva transformación serán consideradas en términos unitarios respecto a su base. En este punto, podemos dar una velocidad de rotación a la “hija”:

```
Final_point = base_transform * T(Mouse_X, Mouse_Y) * R(Time() * 0.001f) *  
S(0.7f, 0.7f) * Initial_point
```

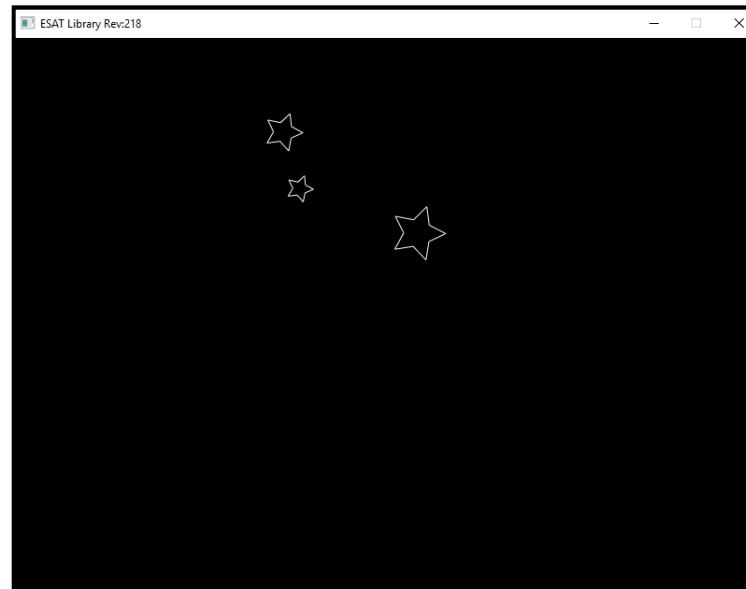


El ratón estaba aquí, este es el centro de la composición.

8ª Transformación: pasos

4. Vamos a repetir la última operación con la estrella “mascota”... y ya tenemos lista la composición final:

```
Final_point = base_transform * daughter_transform * T(Mouse_X, Mouse_Y) *  
              R(Time() * 0.001f) * S(0.7f, 0.7f) * Initial_point
```

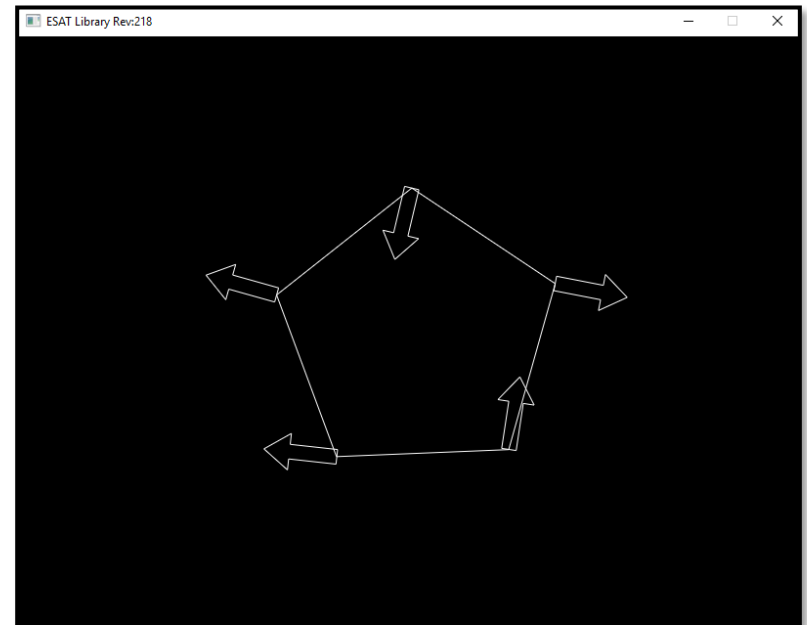


9

9ª Transformación

Programa la siguiente transformación. El pentágono se obtiene con base en el círculo y está trasladado a la posición del ratón, tiene un escalado de “150.0f” unidades en los dos ejes y rota en el sentido de las agujas del reloj. Las flechas, que heredan su transformación, nacen en cada uno de sus vértices, escalan a la mitad y poseen una rotación propia a distinta velocidad. Sus coordenadas son:

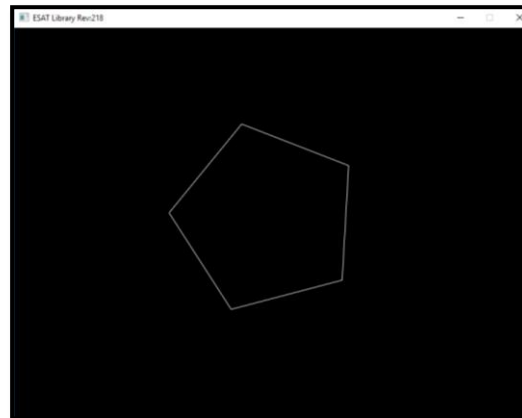
```
float points[10] = {  
    0.0f, 0.4f,  
    0.65f, 0.4f,  
    0.65f, 0.25f,  
    1.0f, 0.5f,  
    0.65f, 0.75f,  
    0.65f, 0.6f,  
    0.0f, 0.6f,  
    0.0f, 0.4f };
```



9ª Transformación: pasos

1. Comenzamos obteniendo el círculo con las funciones trigonométricas dividiendo la circunferencia en cinco partes y aplicando una transformación básica:

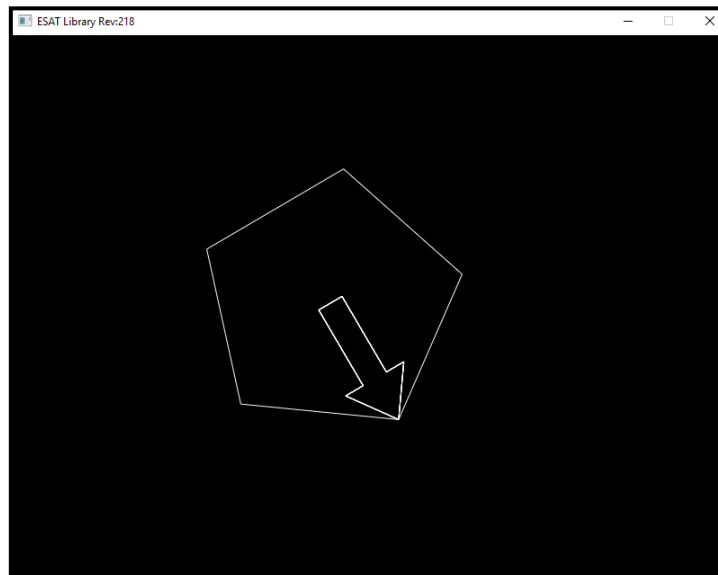
```
for (0 ... kNumberVertices) {  
    pos[...].x = cos((kCircunference / kNumberVertices) * i)  
    pos[...].y = sin((kCircunference / kNumberVertices) * i)  
}  
  
F_point = T(mouse_x, mouse_y) * R(g_angle) * S(150.0f, 150.0f) * I_point
```



9ª Transformación: pasos

2. En segundo lugar, vamos a analizar los puntos de la flecha y vemos que, si queremos que el centro de la figura sea la mitad del borde lateral izquierdo, hay que efectuar una pequeña traslación en el eje de la “Y” para cambiar su centro antes de concatenar la transformación del base:

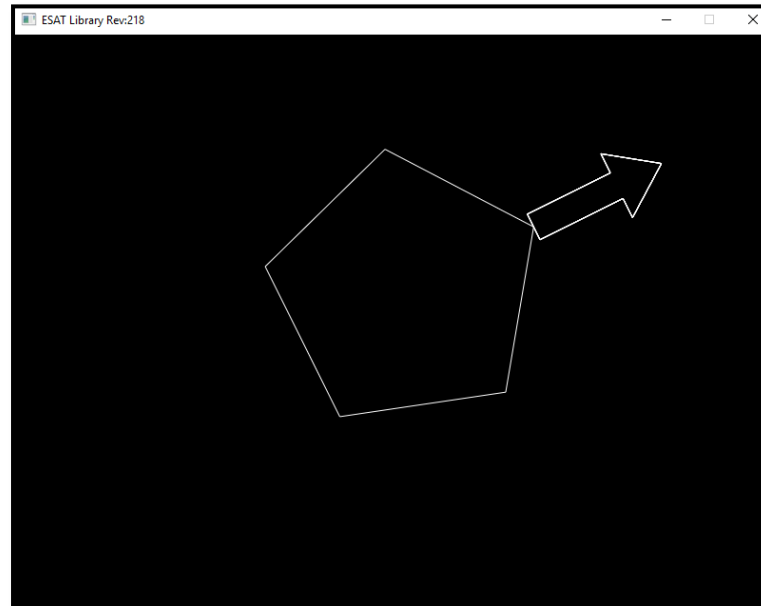
```
Final_point = base_transform * T(0.0f, - 0.5f) * Initial_point
```



9ª Transformación: pasos

3. Adicionalmente, como queremos que aparezca en la punta, vamos a concatenar una nueva traslación, en esta ocasión de una unidad en el eje de la “X”:

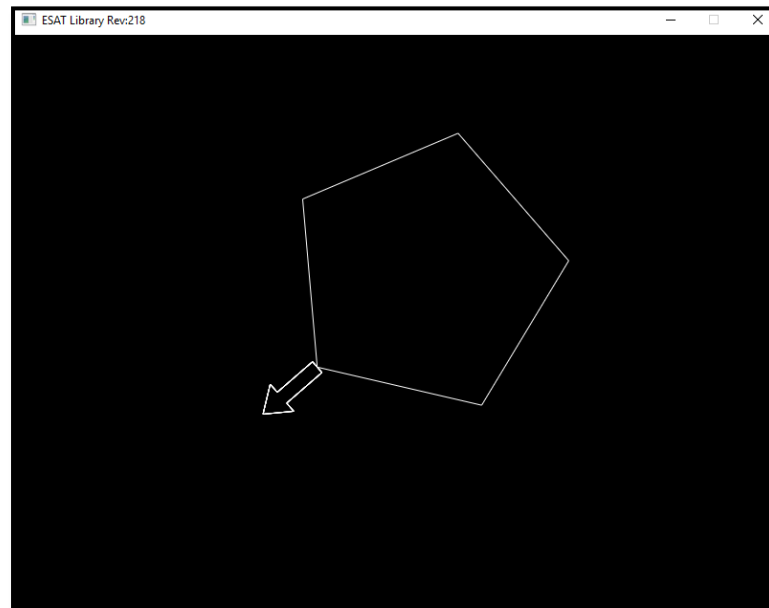
```
Final_point = base_transform * T(1.0f, 0.0f) * T(0.0f, - 0.5f) *  
Initial_point
```



9ª Transformación: pasos

4. ¿Por qué no hemos concatenado ambas traslaciones en una? Porque vamos a añadir algún paso intermedio como un escalado a la mitad:

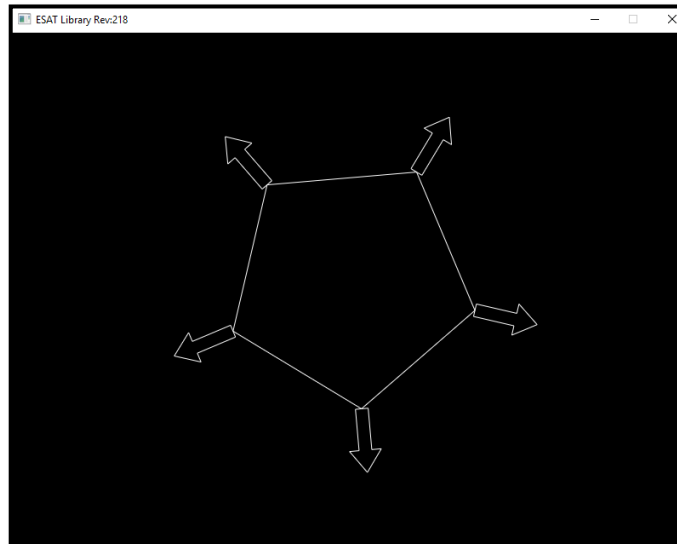
```
Final_point = base_transform * T(1.0f, 0.0f) * S(0.5f, 0.5f) *  
T(0.0f, - 0.5f) * Initial_point
```



9ª Transformación: pasos

5. En este punto, si queremos que nuestras flechas se coloquen cada una en un vértice, nos bastaría con, en un bucle, establecer una pequeña rotación una vez que hemos cambiado (por dos veces) el punto de pivotaje de la figura:

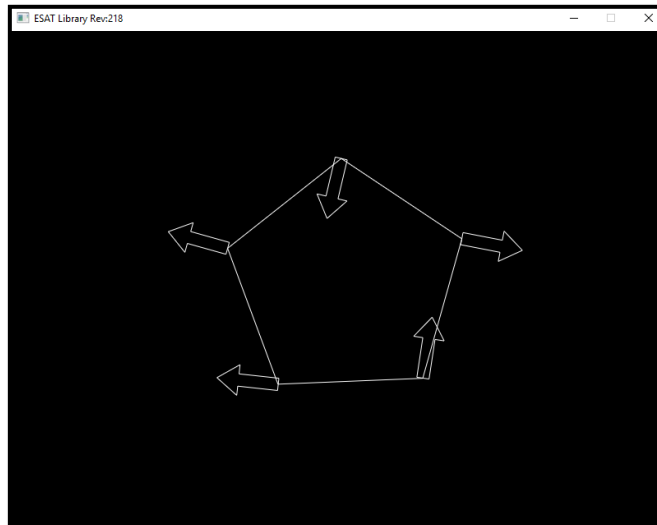
```
Final_point = base_transform * R(Circunference / kNVertices * i) *  
T(1.0f, 0.0f) * S(0.5f, 0.5f) * T(0.0f, - 0.5f) * Initial_point
```



9ª Transformación: pasos

6. Sólo nos quedaría añadir una pequeña rotación antes de realizar el último cambio de pivotaje para que cada flecha rote localmente a una velocidad distinta:

```
Final_point = base_transform * R(Circunference / kNVertices * i) *  
T(1.0f, 0.0f) * R(esat::Time() * 0.001f * i) * S(0.5f, 0.5f) * T(0.0f, - 0.5f)  
* Initial_point
```

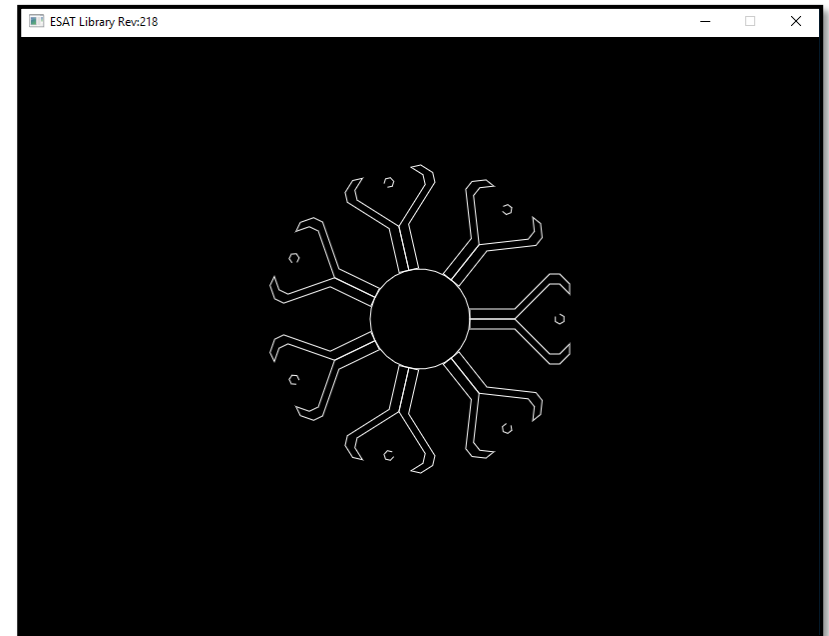


10

10ª Transformación

Consigue la siguiente composición mediante transformaciones. Se te son dados únicamente el siguiente “array” de puntos con coordenadas en dos ejes:

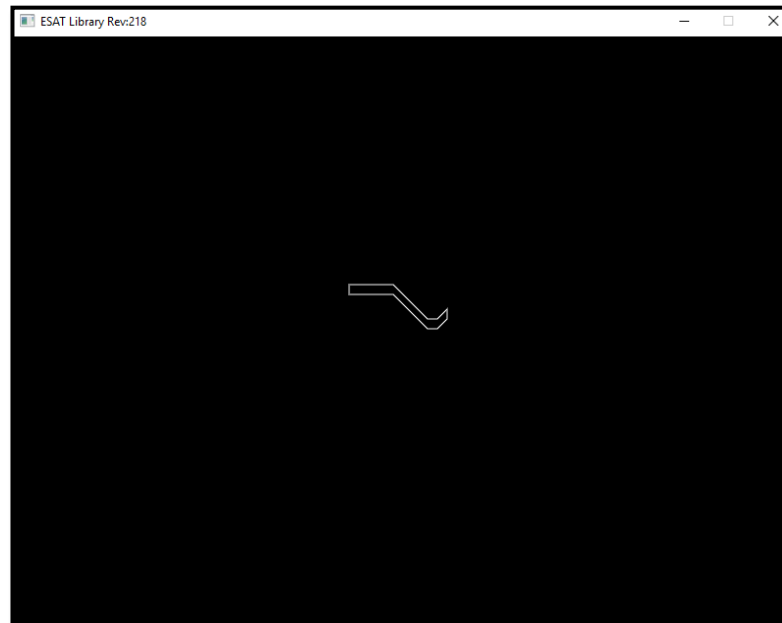
```
float points[20] = {  
    0.0f, 0.0f, 0.45f, 0.0f,  
    0.8f, 0.35f, 0.9f, 0.35f,  
    1.0f, 0.25f, 1.0f, 0.35f,  
    0.9f, 0.45f, 0.8f, 0.45f,  
    0.45f, 0.1f, 0.0f, 0.1f  
};
```



10ª Transformación: pasos

1. En primer lugar, vamos a comenzar representando la figura básica con un escalado de 100 unidades en ambos ejes y una traslación a la posición del ratón.

```
Final_point = T(mouse_x, mouse_y) * S(100.0f, 100.0f) * Initial_point
```

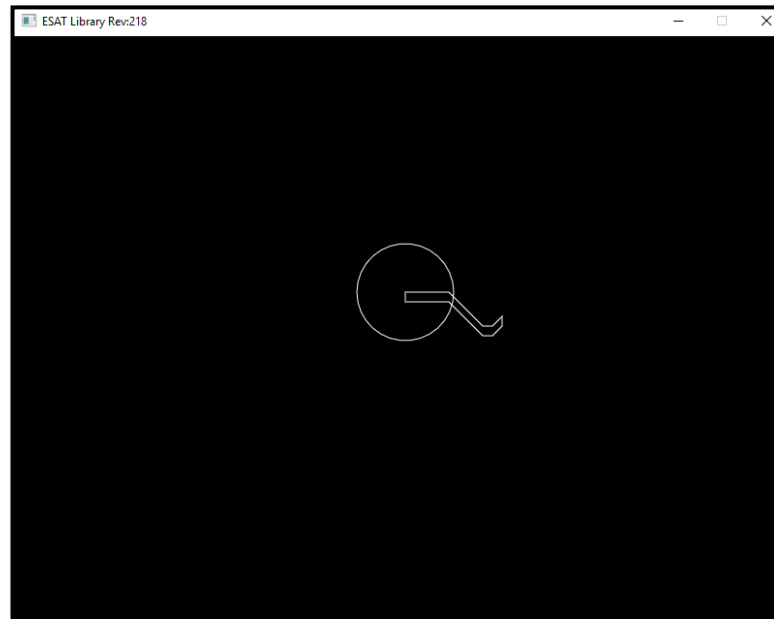


10ª Transformación: pasos

2. Ahora, vamos a representar un círculo con un escalado de 50 unidades y también trasladado a la posición del ratón. No importa si ahora solapamos figuras.

```
Final_point_circle = T(mouse_x, mouse_y) * S(50.0f, 50.0f) * Initial_point
```

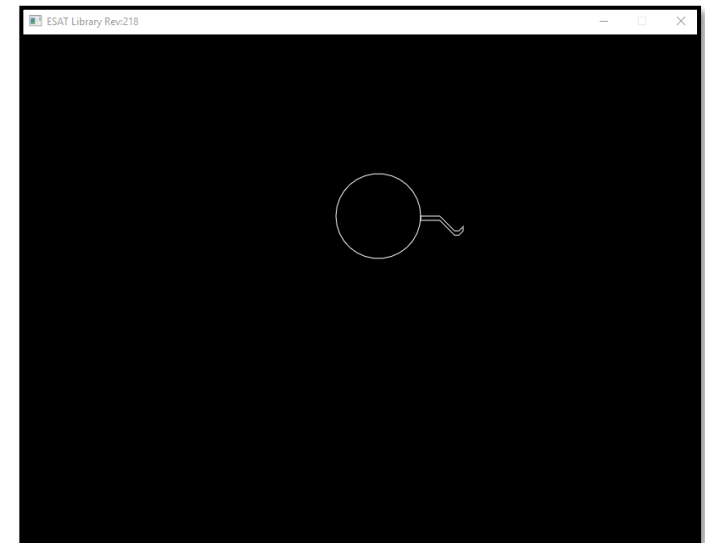
```
Final_point_horn = T(mouse_x, mouse_y) * S(100.0f, 100.0f) * Initial_point
```



10ª Transformación: pasos

3. Bien, para poder relacionar la transformación de ambas figuras geométricas, en el caso del círculo vamos a mantener su transformación pero en el caso del cuerno vamos a hacer que herede la transformación del círculo, van a ser las últimas operaciones que le afecten. Vamos a cambiar el punto de origen local de la figura a (1.0f, 0.0f) para así aprovecharnos de la herencia de transformación y distanciarla en el eje de la 'X' tantas unidades como sea el tamaño de la figura:

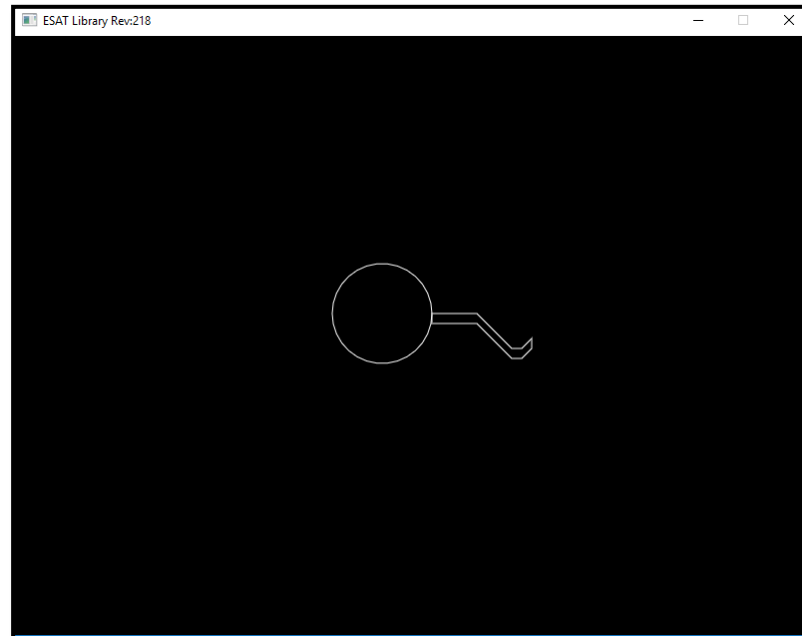
```
Final_point_horn = T(mouse_x, mouse_y) *  
                  S(100.0f, 100.0f) *  
                  T(1.0f, 0.0f) *  
                  Initial_point
```



10ª Transformación: pasos

4. Hecho esto, vamos a escalar al doble la figura hija:

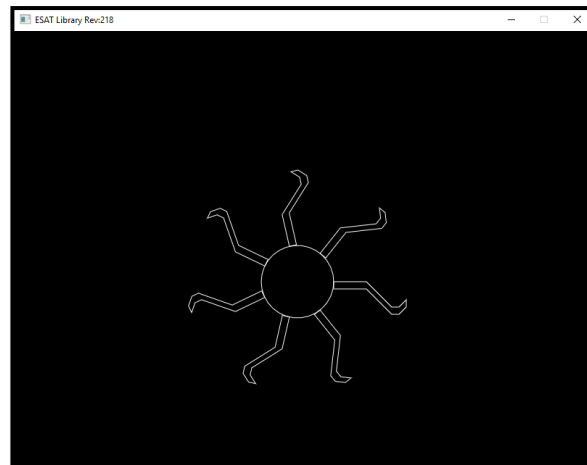
```
Final_point_horn = T(mouse_x, mouse_y) * S(100.0f, 100.0f) * T(1.0f, 0.0f) *  
                  S(2.0f, 2.0f) * Initial_point
```



10ª Transformación: pasos

5. Vamos a colocar siete figuras alrededor del círculo. Para ello vamos a rotar localmente cada figura un ángulo equivalente a la circunferencia entre el número de figuras por la figura actual y también vamos a modificar la traslación local del cuerno mediante funciones trigonométricas que aprovechen el mismo ángulo:

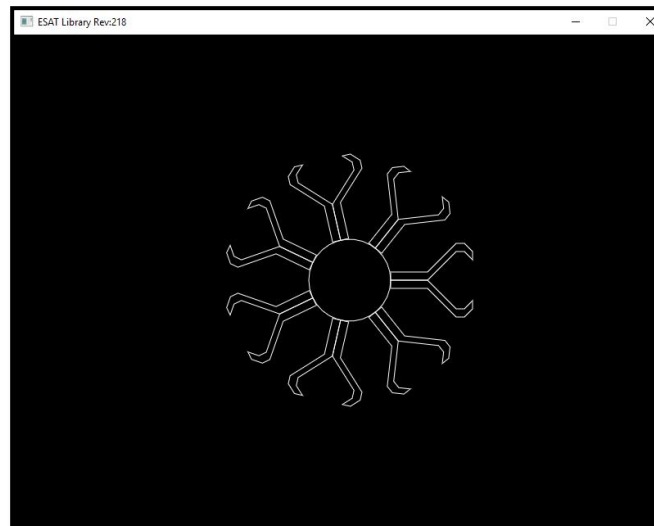
```
Final_point_horn = T(mouse_x, mouse_y) * S(100.0f, 100.0f) *  
T(cos(kCircunference / 7.0f * i), sin(kCircunference / 7.0f * i)) *  
R(kCircunference / 7.0f * i) * S(2.0f, 2.0f) * Initial_point
```



10ª Transformación: pasos

6. Para colocar las siete cornamentas que nos restan, vamos a hacer lo mismo pero con el escalado de la figura hija invertido en el eje de la “y”:

```
Final_point_horn = T(mouse_x, mouse_y) * S(100.0f, 100.0f) *  
T(cos(kCircunference / 7.0f * i), sin(kCircunference / 7.0f * i)) *  
R(kCircunference / 7.0f * i) * S(2.0f, - 2.0f) * Initial_point
```



10ª Transformación: pasos

7. Finalmente, toca representar los hexágonos. Vamos a valernos de la transformación del círculo para heredar de ella. Su transformación local va a estar compuesto de un escalado a la baja (0.1f), de una rotación local equivalente al ángulo de cada par de cornamentas menos 90° (1.57 en radianes) y de una traslación aprovechando el mismo ángulo y multiplicándola por 2.8f, el equivalente a casi tres radios del círculo.

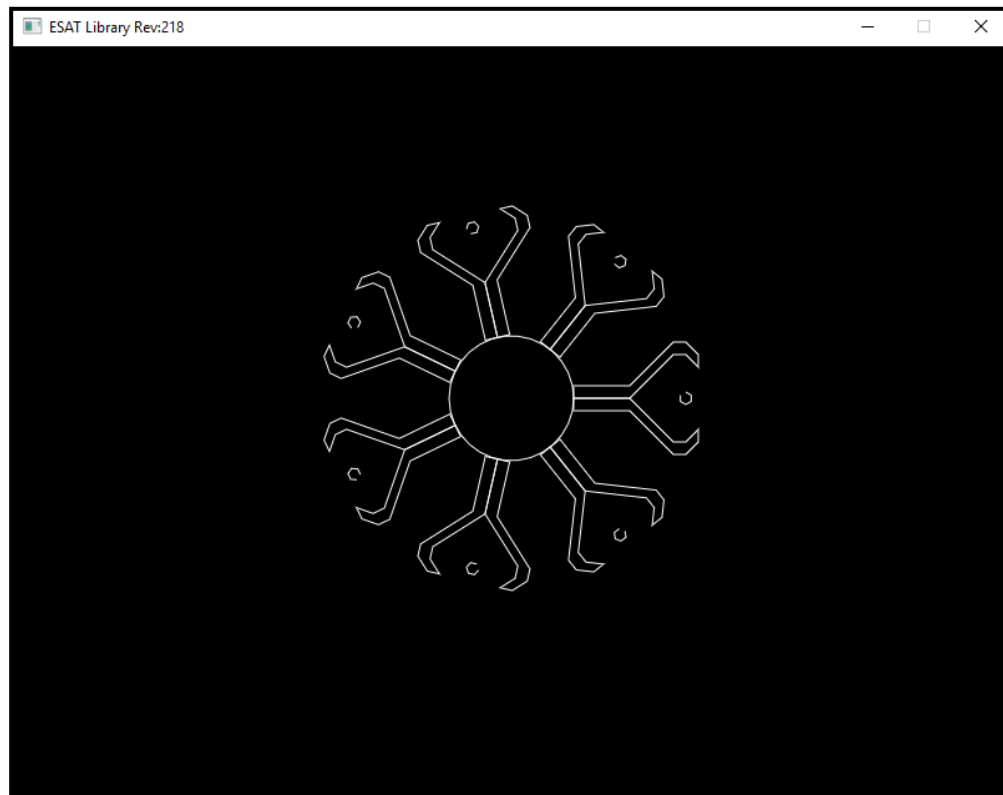
Sabemos esta distancia aproximada porque tomamos el radio del círculo como referencia a la unidad y porque las cornamentas las teníamos escaladas a dos, y si nuestro escalado local es de “0.1f”, sabemos que como máximo vamos a alcanzar una distancia de “2.9” respecto al radio del círculo base del que hereda:

```
float angle = kCircunference / 7.0f * i;

Final_point_horn = T(mouse_x, mouse_y) * S(100.0f, 100.0f) *
T(cos(angle) * 2.8f, sin(angle) * 2.8f) * R(angle - 1.57f) * S(0.1f, 0.1f) *
Initial_point
```

10ª Transformación: pasos

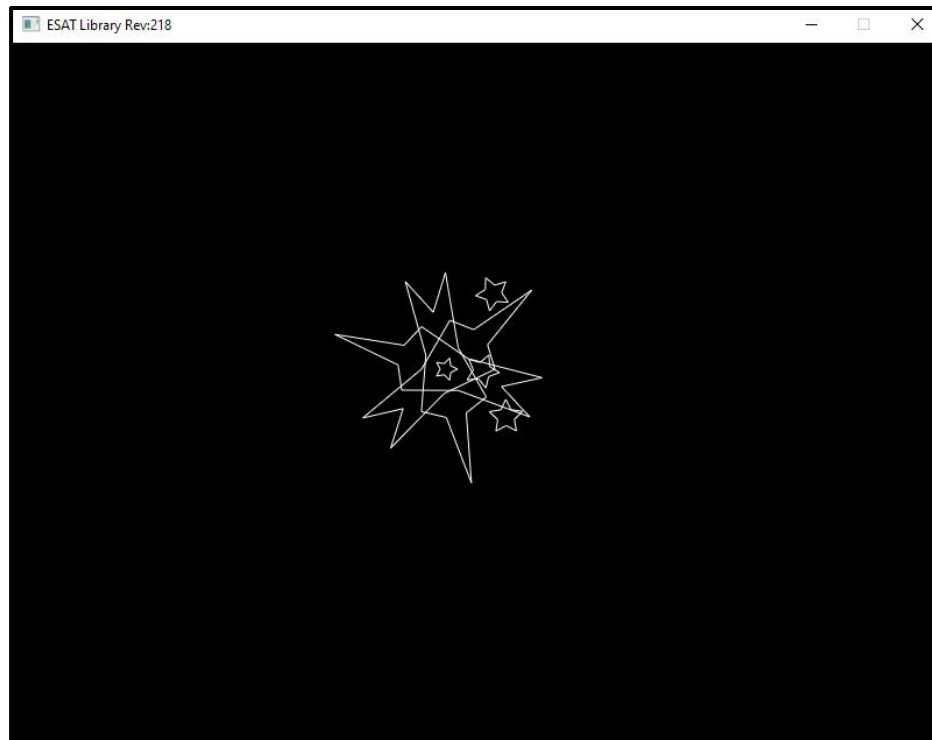
De este modo, ya tenemos la composición final:



11

11ª Transformación

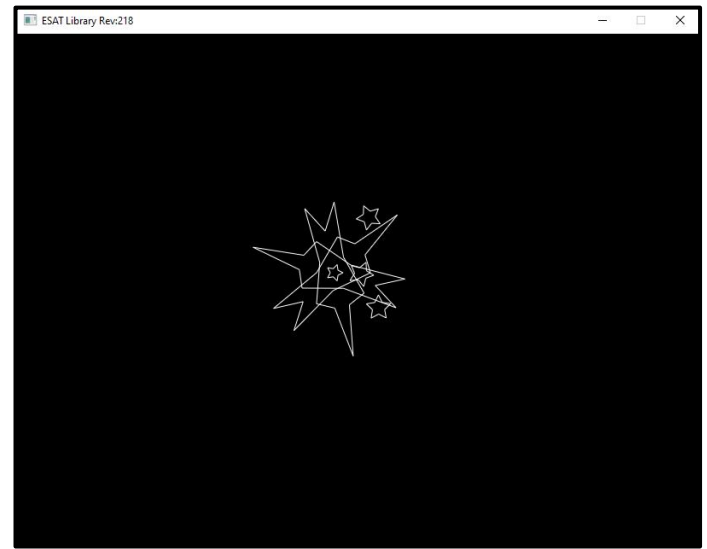
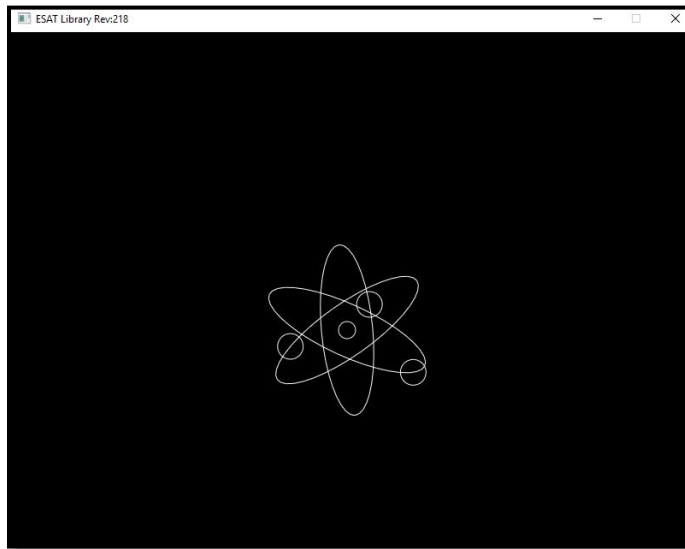
Calcula una estrella básica de cinco puntas y realiza la siguiente composición, dibujando siempre la misma estrella, pero con una transformación diferente cada vez. Las figuras rotarán en el sentido de las agujas del reloj.



11ª Transformación: pasos

1. La resolución de este ejercicio es idéntica a la del ejercicio del neutron, pero con una figura distinta para tratar de despistar:

```
Final_point = base_transform * T(cos(Time() * 0.001f * speed), sin(Time() *  
                                0.001f * speed)) * S(1.0f / 100.0f * 12.0f, 1.0f / 30.0f *  
                                12.0f) * Initial_point
```

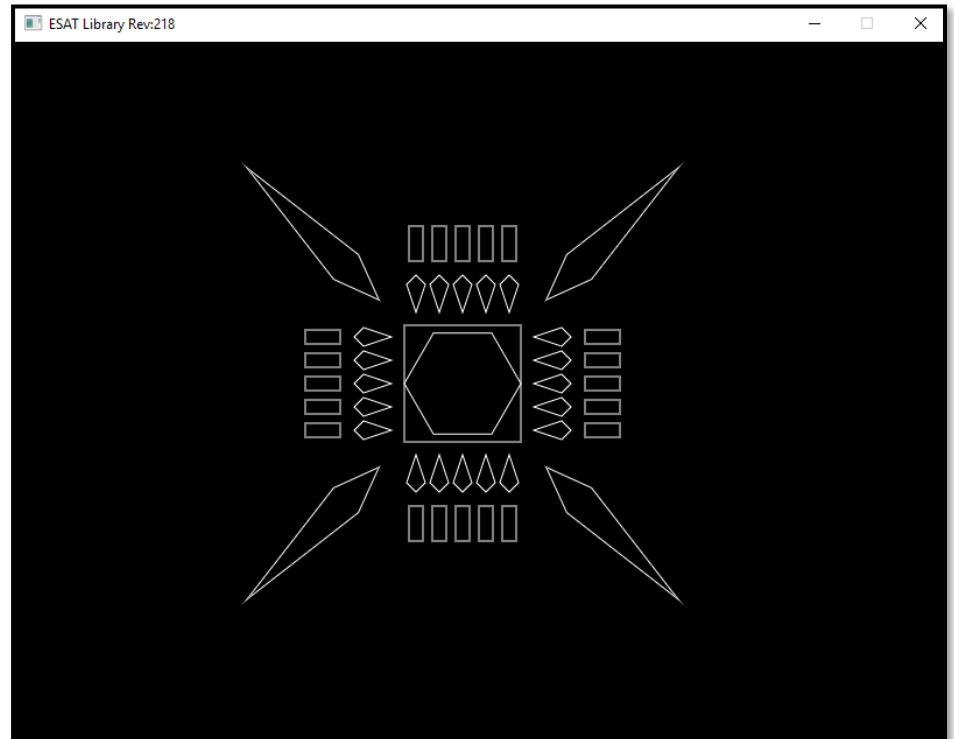


Examen 01

Ex. 01ª Transformación

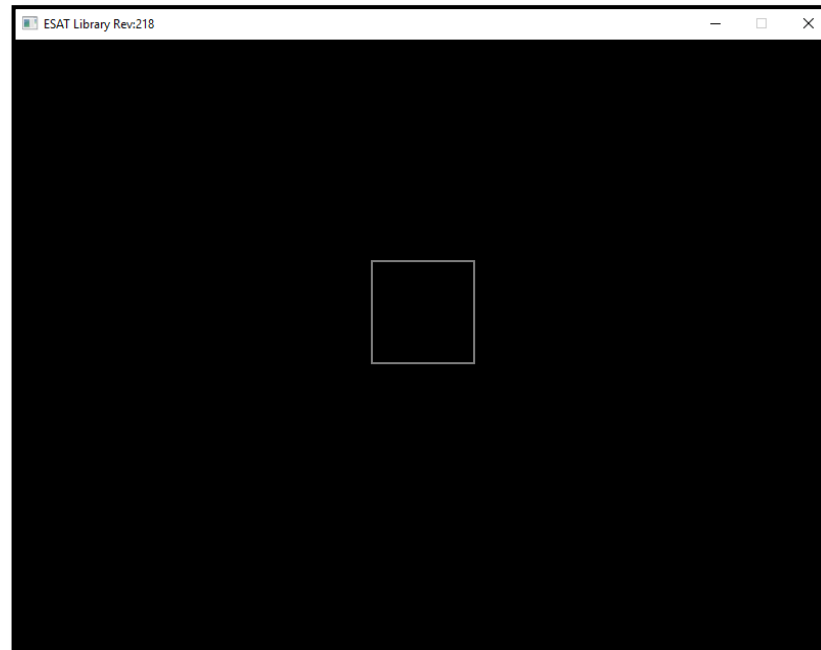
Consigue la siguiente composición mediante transformaciones creándose sí o sí las funciones “DrawArrow” y “DrawQuad”:

```
float arrow[] = {  
    0.0f, 1.0f,  
    1.0f, 0.0f,  
    0.0f, -3.0f,  
    -1.0f, 0.0f };  
  
float square[] = {  
    -1.0f, -1.0f,  
    1.0f, -1.0f,  
    1.0f, 1.0f,  
    -1.0f, 1.0f,  
    -1.0f, -1.0f };
```



01ª Transformación: pasos

1. En este ejercicio, podemos comenzar pintando el cuadrado central con un escalado de “50.0f” unidades en los dos ejes y una traslación a la posición del ratón:

$$\text{Final_point} = T(\text{mouse_x}, \text{mouse_y}) * S(50.0f, 50.0f) * \text{Initial_point}$$


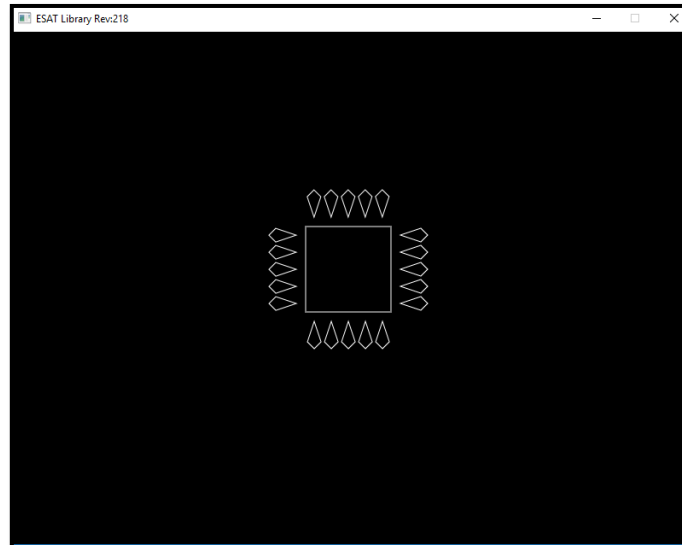
01ª Transformación: pasos

2. Para pintar las flechas interiores puede ser útil valerse de un doble bucle: por una parte para recorrer las filas, y por otro para recorrer las flechas por filas. Emplearemos una rotación para cambiar su orientación y otra para cambiar el cuadrante con ayuda de una traslación que modifique el punto de pivotaje. La transformación que le aplicaremos para renderizar cada sección es la siguiente:

```
for (i ... kNRows) {  
    for (j ... kNCols) {  
        Final_point = T(mouse_x, mouse_y) * R(6.28f / 4.0f * j) *  
            T(85.0f, -40.0f + 20.0f * i) * R(6.28f / 4.0f * 3.0f) *  
            S(8.0f, 8.0f) * Initial_point  
    }  
}
```

01ª Transformación: pasos

```
for (i ... kNRows) {  
    for (j ... kNCols) {  
        Final_point = T(mouse_x, mouse_y) * R(6.28f / 4.0f * j) *  
            T(85.0f, -40.0f + 20.0f * i) * R(6.28f / 4.0f * 3.0f) *  
            S(8.0f, 8.0f) * Initial_point  
    }  
}
```



01ª Transformación: pasos

3. Vamos a emplear el mismo planteamiento para los cuadrados, sólo que a una distancia mayor y a un escalado distinto:

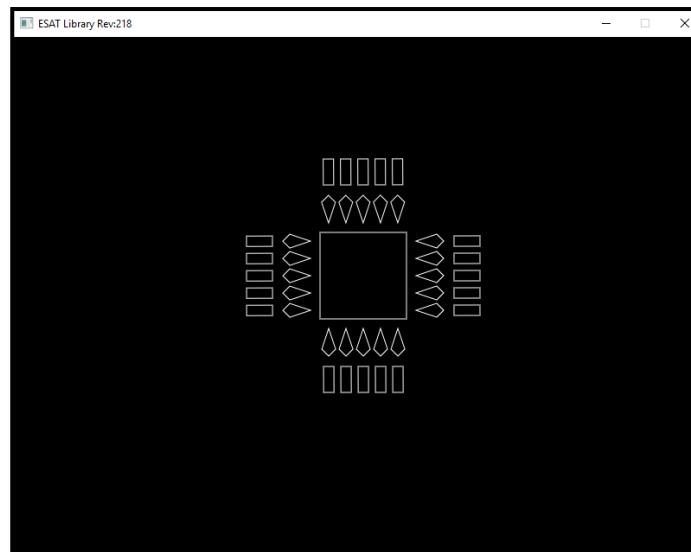
```
for (i ... kNRows) {  
    for (j ... kNCols) {  
        Final_point = T(mouse_x, mouse_y) * R(6.28f / 4.0f * j) *  
            T(120.0f, -40.0f + 20.0f * i) * R(6.28f / 4.0f * 3.0f) *  
            S(6.0f, 15.0f) * Initial_point  
    }  
}
```

01ª Transformación: pasos

```

for (i ... kNRows) {
    for (j ... kNCols) {
        Final_point = T(mouse_x, mouse_y) * R(6.28f / 4.0f * j) *
                        T(120.0f, -40.0f + 20.0f * i) * R(6.28f / 4.0f * 3.0f) *
                        S(6.0f, 15.0f) * Initial_point
    }
}

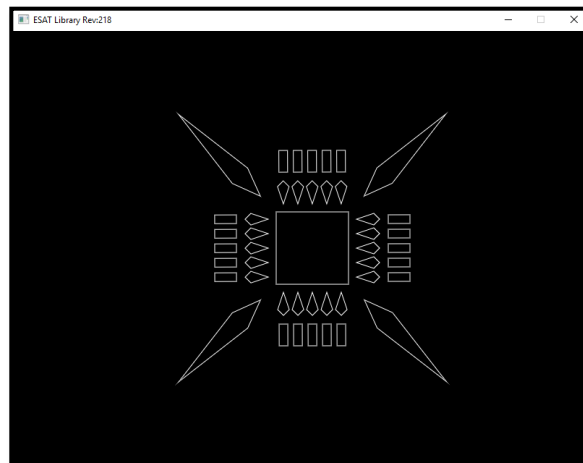
```



01ª Transformación: pasos

4. Hacemos lo mismo con las flechas diagonales, escalándola de modo dispar en ambos ejes y rotándola inicialmente 225° (3.9269 radianes):

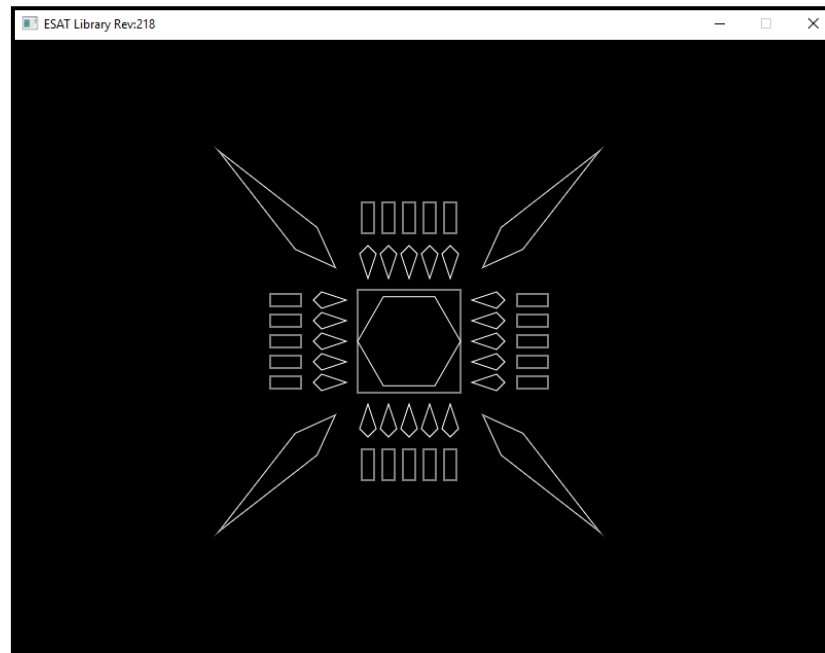
```
for (i ... kNRows) {  
    Final_point = T(mouse_x, mouse_y) * R(6.28f / 4.0f * i) *  
        T(-100.0f, 100.0f) * R(3.9269) * S(15.0f, 40.0f) *  
        Initial_point  
}
```



01ª Transformación: pasos

5. En último lugar, representamos el pentágono que nos queda y ya tendríamos la figura:

```
Final_point = T(mouse_x, mouse_y) * S(50.0f, 50.0f) * Initial_point
```

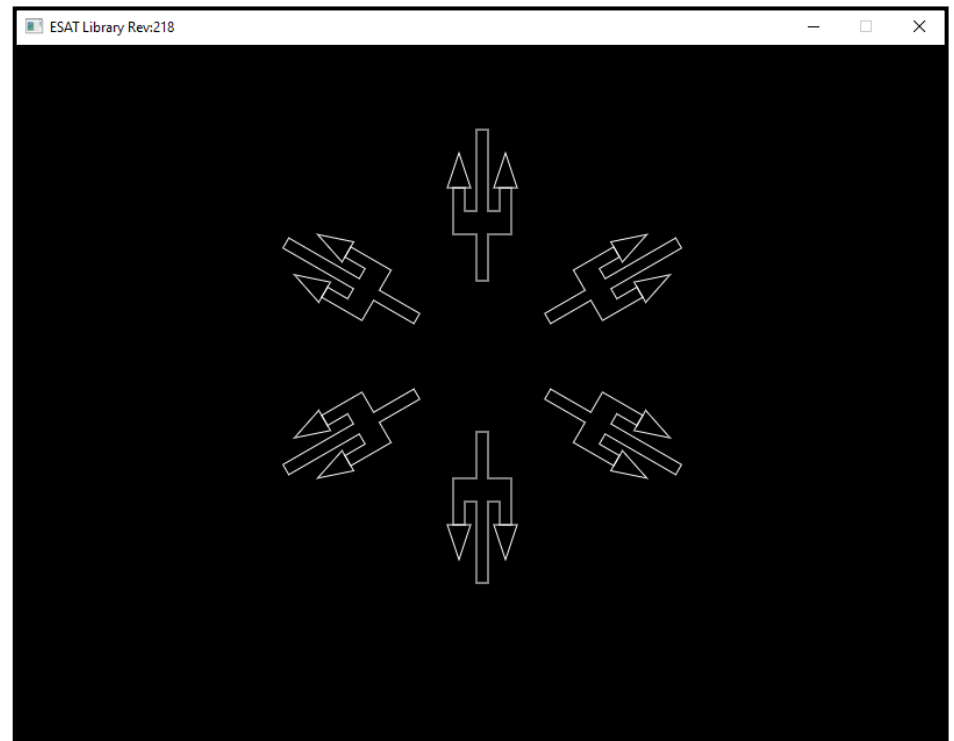


Examen 02

Ex. 02ª Transformación

Consigue la siguiente composición mediante transformaciones creándose sí o sí las funciones “DrawTriangle” y “DrawTrident”. Las coordenadas de los puntos de cada figura son las siguientes:

```
float triangle[] = {  
    -1.0f, 0.0f,  
    0.0f, 1.0f,  
    0.0f, 3.0f };  
  
float trident[] = {  
    siguiente página  
};
```



02ª Transformación: pasos

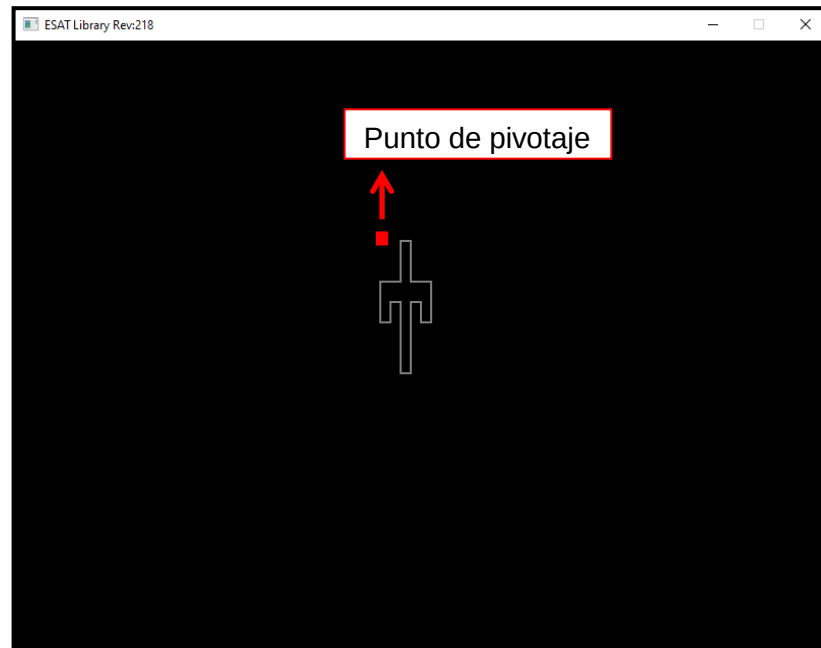
Consigue la siguiente composición mediante transformaciones creándose sí o sí las funciones “DrawTriangle” y “DrawTrident”. Las coordenadas de los puntos de cada figura son las siguientes:

```
float trident[] = {  
    2.0f, 0.0f,          ...  
    3.0f, 0.0f,          3.0f, 13.0f,  
    3.0f, 4.0f,          2.0f, 13.0f,  
    5.0f, 4.0f,          2.0f, 6.0f,  
    5.0f, 8.0f,          1.0f, 6.0f,  
    4.0f, 8.0f,          1.0f, 8.0f,  
    4.0f, 6.0f,          0.0f, 8.0f,  
    3.0f, 6.0f,          0.0f, 4.0f,  
    ...                  2.0f, 4.0f };
```

02ª Transformación: pasos

1. En esta ocasión, se observa que puede haber una dependencia jerárquica de los triángulos con respecto al tridente, por lo que puede ser aconsejable comenzar pintando un solo tridente con una transformación básica para obtener información:

```
Final_point = T(mouse_x, mouse_y) * R(0.0f) * S(10.0f, 10.0f) * Initial_point
```



02ª Transformación: pasos

2. Una vez visto que el punto de pivotaje se encuentra en la esquina superior izquierda, tratamos de centrarlo en el eje de la “X” con una traslación inicial.

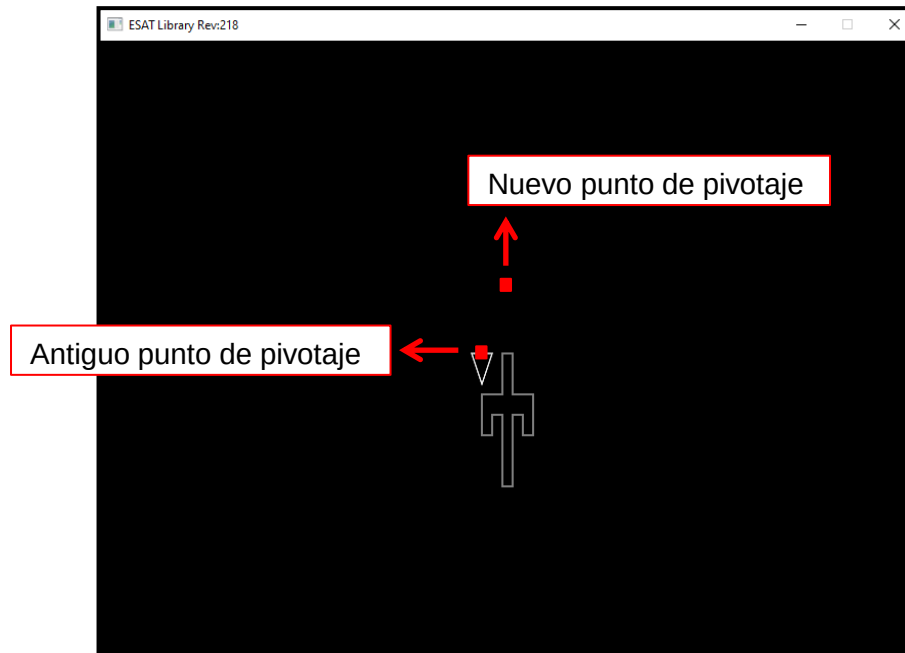
```
Final_point = T(mouse_x, mouse_y) * R(0.0f) * S(10.0f, 10.0f) *  
T(-2.5f, 8.0f) * Initial_point
```



02ª Transformación: pasos

3. Si ahora tratamos de pintar los triángulos aprovechándonos de la matriz base, vemos que ambos se pintan en la misma posición y con respecto al punto de pivotaje original.

```
Final_point = base_matrix * Initial_point
```



02ª Transformación: pasos

4. Aquí hay que tomar en cuenta las dimensiones de la figura original para saber cuánto trasladarlos.

```
Final_point_triangle_1 = base_matrix * T(0.5f, 8.0f) * Initial_point
```

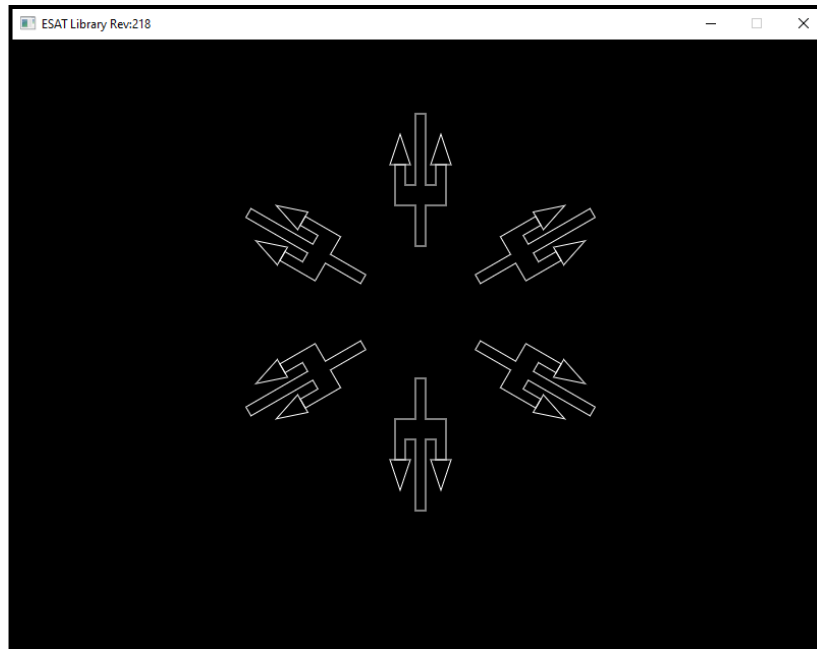
```
Final_point_triangle_2 = base_matrix * T(4.5f, 8.0f) * Initial_point
```



02ª Transformación: pasos

5. Una vez hecho esto, sólo quedaría pintarlos en un bucle modificando la rotación de la figura base:

```
Final_point = T(mouse_x, mouse_y) * R(angle * i) * S(10.0f, 10.0f)  
* Initial_point
```

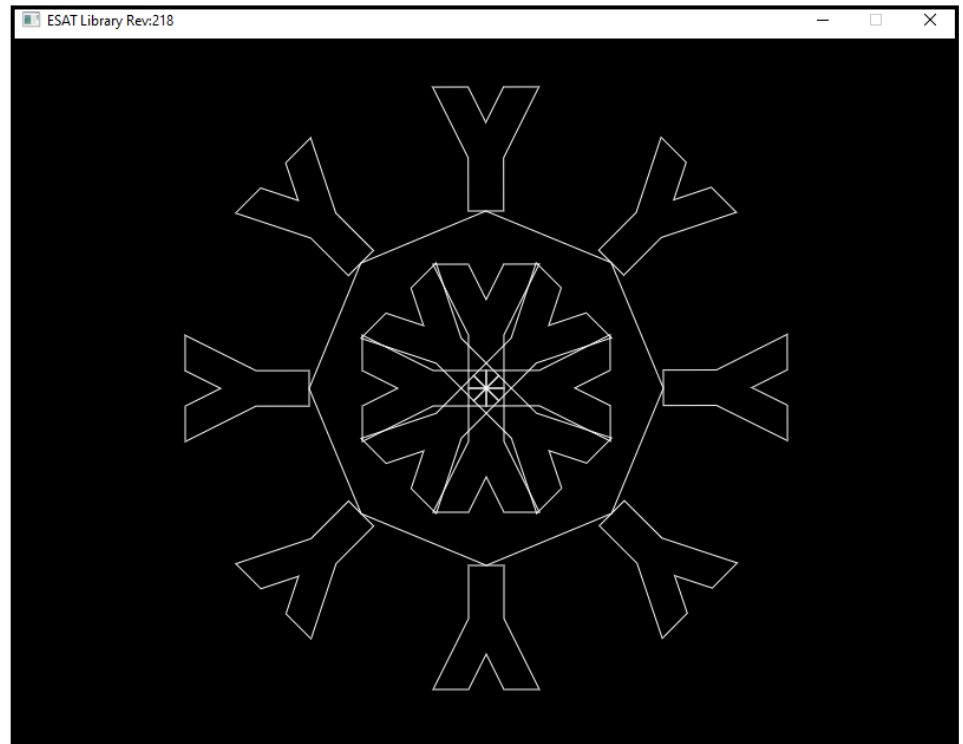


Examen 03

Ex. 03ª Transformación

Consigue la siguiente composición mediante transformaciones creándose sí o sí las funciones “DrawCircle” y “DrawPolygon”. La circunferencia se ha calculado de la forma habitual, mientras que las coordenadas de la otra figura son:

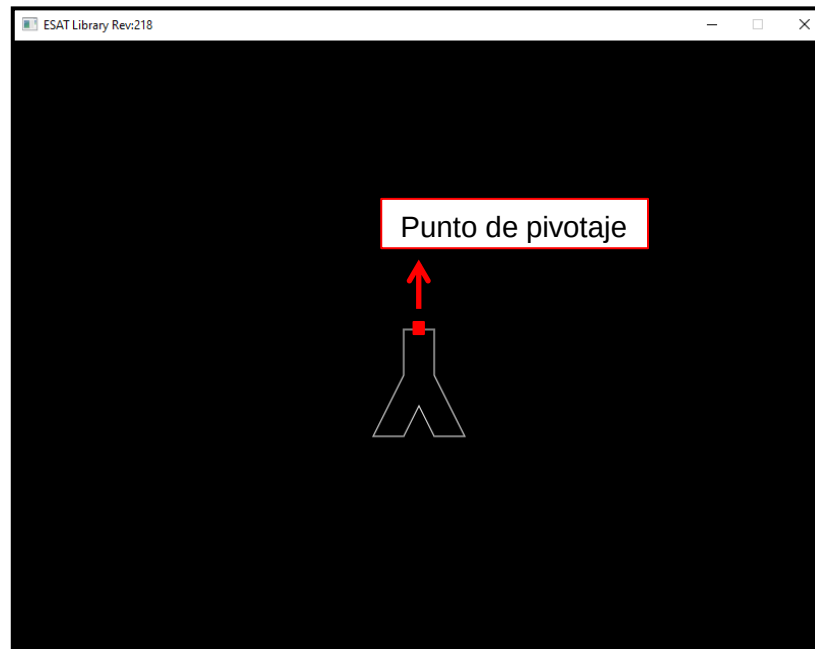
```
float figure[] = {  
    1.0f, 0.0f,  
    1.0f, 3.0f,  
    3.0f, 7.0f,  
    1.0f, 7.0f,  
    0.0f, 5.0f,  
    -1.0f, 7.0f,  
    -3.0f, 7.0f,  
    -1.0f, 3.0f,  
    -1.0f, 0.0f };
```



03ª Transformación: pasos

1. Lo primero sería realizar un pintado de la figura para obtener información de ella. Como la figura original se encontraba entre “-3.0f” y “3.0f” para el eje de la “X” y “0.0f” y “7.0f” para la “Y”, se escala a “15.0f” para controlar su tamaño entre “-45.0f” y “45.0f” para la “X” y “0.0f” y “105.0f” para la “Y”:

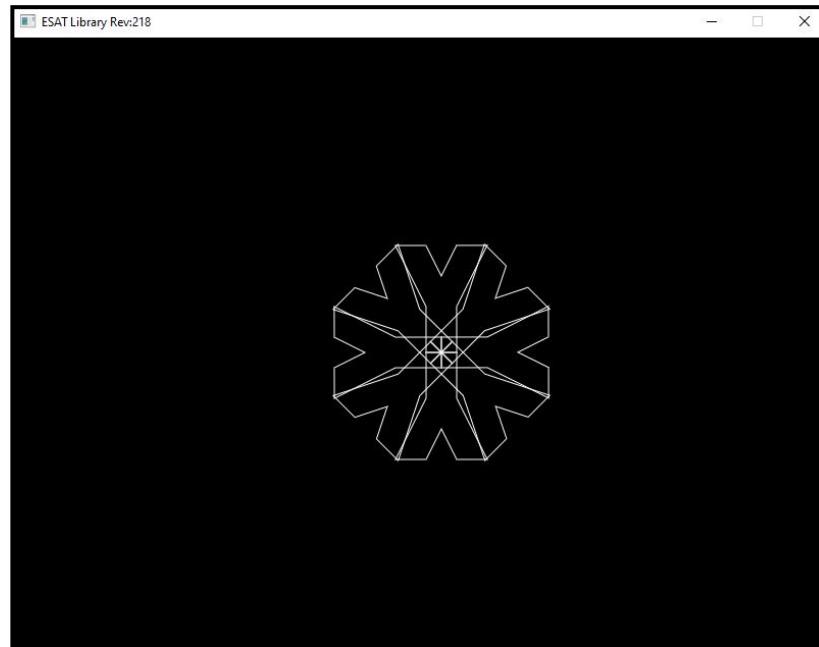
```
Final_point = T(mouse_x, mouse_y) * R(0.0f) * S(15.0f, 15.0f) * Initial_point
```



03ª Transformación: pasos

2. Si con eso se realiza un bucle de iteraciones igual al número de vértices del círculo a pintar, ya se pueden ver todas las figuras del anillo interior:

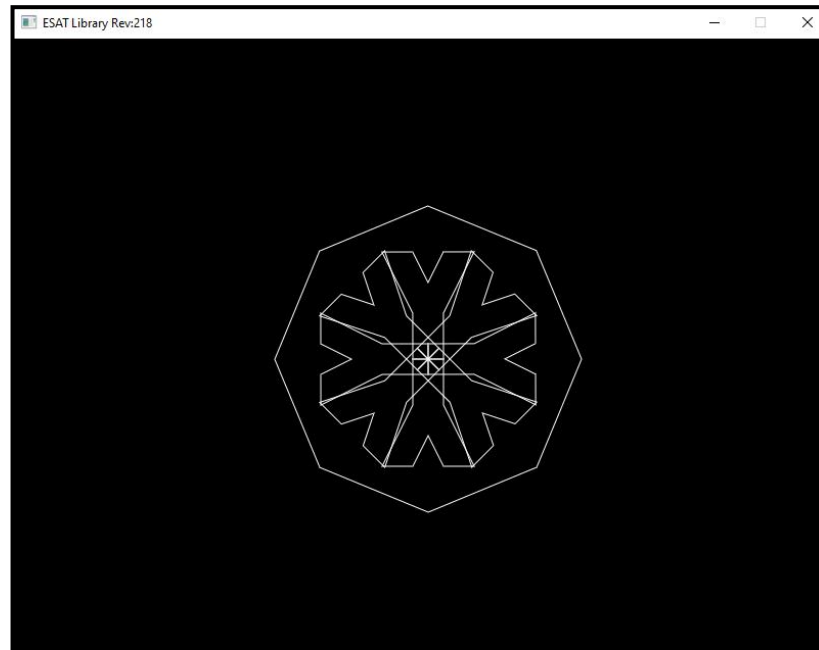
```
Final_point = T(mouse_x, mouse_y) * R(angle * i) * S(15.0f, 15.0f)  
* Initial_point
```



03ª Transformación: pasos

3. A continuación puede ser buena idea pintar el círculo exterior con una transformación que tenga en cuenta las dimensiones de las figuras del anillo interior...

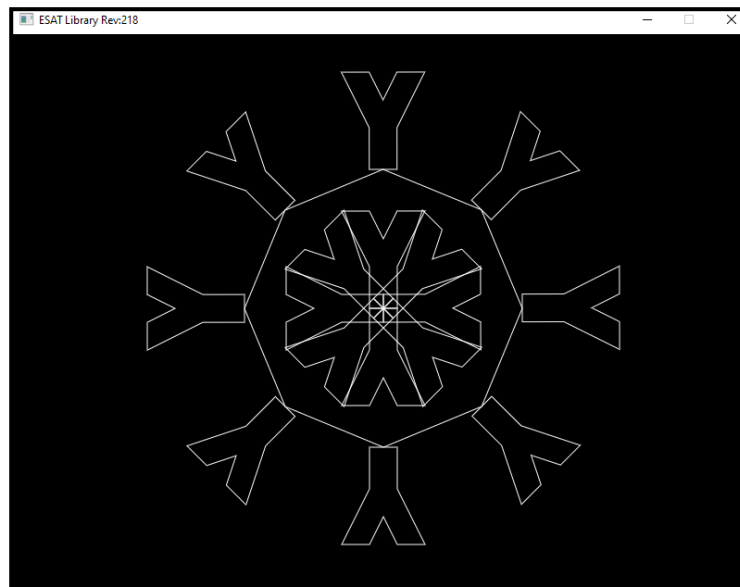
```
Final_point = T(mouse_x, mouse_y) * R(0.0f) * S(150.0f, 150.0f)* Initial_point
```



03ª Transformación: pasos

4. ... para aprovechar esa matriz base y cada uno de los puntos del círculo como nuevo punto de anclaje de cada figura del anillo exterior. Para respetar los escalados, eso sí, tal vez convenga anular el escalado intrínseco del círculo:

```
Final_point = base_matrix * R(angle * i) * T(0.0f, 1.0f) * S(1.0f / 150.0f  
* 15.0f, 1.0f / 150.0f * 15.0f) * Initial_point
```



IMPORTANTE

Repito: todo el código mostrado en este documento de respuestas a los ejercicios de clase es pseudocódigo y sólo representa los pasos necesarios para renderizar cada una de las siguientes transformaciones.



www.esat.es